

Hochschule Esslingen

Fakultät für Informatik

Masterarbeit

Prognose individuellen Mobilitätsverhaltens auf Basis eines mit Bewegungsdaten trainierten Machine-Learning-Modells

Achim Baumgärtner

Matrikelnummer: 766010

Erstgutachter: Prof. Dr. Sonntag

Zweitgutachter: Prof. Dr. Schober

Esslingen am Neckar, 28. Juli 2026

Kurzfassung

Die Auswertung fahrzeugspezifischer Daten ermöglicht es, bei kontinuierlichen Nutzungsmustern das Bewegungsverhalten vorherzusagen. Dadurch kann bidirektionales Laden effizienter eingesetzt werden, was sowohl die Fahrzeughalter als auch das Stromnetz entlastet. Ziel ist es, durch Machine Learning ein Modell zu entwerfen, das möglichst genau individuelles Verkehrsverhalten prognostizieren kann. Zu bewerkstelligen ist dieses Vorhaben durch eine Ablationsstudie; Es werden auf Basis einer Datengrundlage Modelle erstellt und untereinander verglichen, wodurch die Annahme entsteht, dass ein Modell besser abschneiden wird, als die anderen. Das stärkste Modell wird dann weiter entwickelt um die Ergebnisse zu optimieren und somit die Zuverlässigkeit und den Beitrag zum Stromnetz von elektrischen Fahrzeugen zu erhöhen.

Schlüsselwörter: Künstliche Intelligenz, Machine Learning, Prognose, Fahrzeugnutzung, Elektromobilität, Bidirektionales Laden, Random Forest, LightGBM

Inhaltsverzeichnis

Kurzfassung	i
Abbildungsverzeichnis	iv
Tabellenverzeichnis	vi
1. Einleitung	1
1.1. Motivation	1
1.2. Problemstellung	2
1.3. Zielsetzung	3
1.4. Forschungsfragen	3
1.5. Aufbau der Arbeit	5
2. Theoretische Grundlagen	6
2.1. Mobilitätsdaten	6
2.2. Verarbeitung von Mobilitätsdaten	7
2.3. Machine Learning	9
2.3.1. Überwachtes Lernen	9
2.3.2. Merkmale und Feature Engineering	11
2.3.3. Training	12
2.3.4. Generalisierung	12
2.3.5. Validierung	13
2.3.6. Bewertungsmetriken	14
2.3.7. Entscheidungsbäume	14
2.3.8. Random Forest	17
2.3.9. Light Gradient Boosting Machine	18
2.3.10. Unsicherheit und Quantilsregression	18
2.3.11. Long Short-Term Memory	19
2.4. Stand der Forschung	20
2.4.1. Verwandte Arbeiten	21
3. Methodik	28
3.1. Forschungsdesign	28

3.2. Untersuchungsablauf	30
3.3. Datengrundlage und Quellen	32
3.4. Feature Engineering	35
3.5. Modellauswahl	36
3.6. Experimentaldesign	37
3.7. Bewertung	39
4. Implementierung	42
4.1. Anforderungen	42
4.2. Systemarchitektur	45
4.3. Technologieauswahl	47
4.4. Umsetzung	49
4.5. Herausforderungen	53
5. Evaluation	55
5.1. Evaluationsdesign	55
5.2. Testumgebung	56
5.3. Ergebnisse	57
6. Diskussion	62
6.1. Interpretation der Ergebnisse	62
6.2. Vergleich mit verwandten Arbeiten	62
6.3. Limitationen	62
6.4. Implikationen	63
7. Fazit und Ausblick	64
7.1. Beantwortung der Forschungsfragen	64
7.2. Ausblick	64
Literaturverzeichnis	65
A. Anhang	68
A.1. Zusätzliche Abbildungen	68
A.2. Zusätzliche Tabellen	70
A.3. Quellcode	70
Eidesstattliche Erklärung	71

Abbildungsverzeichnis

2.1.	Beispielhafte Auszüge zweier Mobilitätsdatensätze.	6
2.2.	Generalisierte Pipeline zur Verarbeitung von Mobilitätsdaten.	7
2.3.	Teilbereiche des maschinellen Lernens mit beispielhaften Anwendungen (Wuttke 2024).	10
2.4.	Entscheidungsbaum mit Aufteilungskriterien (nach Schilli 2017).	15
2.5.	Bagging: Aus den Trainingsdaten werden N Bootstrap-Stichproben gezogen, auf denen jeweils ein Baum trainiert wird; deren Vorhersagen werden zu einem Ensemble aggregiert (ResearchGate 2022).	16
2.6.	Boosting: Die Modelle werden nacheinander trainiert, wobei jedes folgende Modell auf den Fehlern des vorherigen Modells basiert (Re- searchGate 2021).	17
2.7.	Baumwachstum in Ensemble-Verfahren (ScienceDirect 2026).	17
2.8.	(a) Heteroskedastisches Prognoseband: Median (P50) und 80 %-Intervall (P10–P90), dessen Breite mit der Streuung wächst. (b) ρ_τ für $\tau =$ 0,1/0,5/0,9; die Asymmetrie schiebt die Schätzung an das jeweilige Quantil (Koenker u. a. 1978).	19
2.9.	Aufbau einer LSTM-Zelle: Der Zellzustand C_t wird durch Forget-, Input- und Output-Gate gesteuert, die per σ - und tanh-Aktivierung festlegen, was aus dem Gedächtnis verworfen, neu geschrieben und ausgelesen wird (dida 2026).	20
2.10.	Verteilung der Fahrstanz (a) und der Abfahrtszeit des ersten Tages- trips (b) nach Wochentag (Lindroth u. a. 2024).	23
2.11.	Zeitliche Muster der Ladeanfragen im verwendeten Datensatz (Sanami u. a. 2025).	25
3.1.	Variablenstruktur des Forschungsdesigns: zwei veränderliche unab- hängige Variablen (Daten, Algorithmus) und eine abhängige Variable (Prognosegüte). Konstante Hyperparameter und Trainingsprotokolle wirken als Kontrollvariablen.	29
3.2.	Schematischer Workflow der Methodik: von den fünf Datenquellen über den geteilten Feature-Kern und die beiden Aufgabenmodelle bis zum Experimentaldesign und der Bewertung.	31

4.1.	Statische Komponentenstruktur: Datenadapter, Feature-Engineering, Klassifikator, Forecaster, Persistenz und Schnittstellen.	45
4.2.	Dynamischer Ablauf eines Forecast-Requests vom Frontend über das Backend und Persistenz bis zum Forecaster und dessen Aufbereitung und Darstellung der Ergebnisse.	46
4.3.	Persistenzlayout des Systems: Modelle und Vorhersagen werden in einer hierarchischen Verzeichnisstruktur gespeichert, die eine einfache Navigation und Verwaltung der Daten ermöglicht. Modelle liegen gesondert von den Vorhersagen, die in Unterverzeichnissen nach Modell, Run und Zeitstempel organisiert sind.	47
5.1.	5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (RandomForest)	58
A.1.	5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (LightGBM) . .	68
A.2.	5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (LSTM, tabellarische Repräsentation)	69
A.3.	5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (LSTM, sequentielle Repräsentation)	69

Tabellenverzeichnis

1.1. Forschungsfragen der Arbeit.	4
2.1. Phasen der Mobilitätsdaten-Pipeline	8
2.2. Charakterisierung von Anpassungen anhand von Fehlerwerten Géron 2022.	13
2.3. Zentrale Forschungssektionen im Umfeld der individuellen Fahrzeug- nutzungsprognose.	21
2.4. Fünf Fahrertypen des Individualverkehrs nach Ji u. a. (2023).	27
3.1. Hypothesen zu den Forschungsfragen.	30
3.2. Übersicht der fünf Datenquellen und ihrer identifizierenden Eigen- schaften.	32
4.1. Anforderungen an Daten und System	43
4.2. Anforderungen an die Machine-Learning-Modelle	44
5.1. Modell (RandomForest) im Vergleich zu naiven Baselines, In-Distribution je Datenquelle auf demselben zeitlichen Test-Split. Majority = „immer geparkt“; Persistenz-168 = gleiche Stunde der Vorwoche; Klimatologie = $P(\text{fahren} \mid \text{Wochentag}, \text{Stunde})$ aus dem Trainingsteil.	57
5.2. In-distribution (in eigener Menge – Datenquelle, in diesem Fall) Güte- maße des Driving-Classifiers je Datenquelle. RandomForest, LightGBM und tab. LSTM-Variante nutzen Merkmalssatz; seq. Variante sieht Rohfenster. „n. z.“ = nicht als Trainingsquelle zugelassen, da Quelle Mindestkriterium von 100 Fahrzeugwochen verfehlt (Abschnitt 3.5).	59
5.3. Ablation der Merkmalsgruppen: PR-AUC je Datenquelle, zeitlich in- distribution. Verworfenen Zeilen richten sich in allen Varianten nach dem vollständigen Merkmalssatz, sodass je Quelle identische Beob- achtungen zugrunde liegen.	61

1. Einleitung

Die Elektromobilität ist ein immer wichtiger werdendes Thema. Sie trägt zur Reduzierung von Treibhausgasemissionen bei, verringert die Abhängigkeit zu fossilen Brennstoffen und somit auch wirtschaftliche Abhängigkeiten. Gleichzeitig bietet sie eine neue Möglichkeit bei der Energieversorgung.

Die Möglichkeiten bei der Energieversorgung ist hier besonders interessant. Fahrzeuge, die mit Strom betrieben werden, können nicht nur als Verbraucher betrachtet werden, sondern auch als Energiespeicher. Elektrofahrzeuge können also nicht nur Energie aufnehmen, sondern diese auch wieder abgeben, was besonders im Kontext von erneuerbaren Energien von großer Bedeutung ist.

Im Moment bestehen noch kaum rentable Möglichkeiten, überschüssige Energie aus erneuerbaren Quellen zu speichern, weshalb dem Thema eine besondere Relevanz zukommt: Durch eine zuverlässige Speicherung könnte Energie auch dann genutzt werden, wenn Solar- oder Windanlagen nicht aktiv erzeugen. Damit ließe sich der Anteil erneuerbarer Energien an der Versorgung deutlich erhöhen, was ein wichtiger Beitrag zur Energiewende wäre.

Jedoch bringt eine Speicherung von Energie in Elektrofahrzeugen einen geringfügigen Mehrwert - ökonomisch wie auch ökologisch - wenn diese nicht zuverlässig gesteuert wird. Es wird ein System benötigt, das die intelligente Steuerung von Energieflüssen zwischen Fahrzeug und Stromnetz ermöglicht. Eine intelligente Steuerung ist nur möglich, wenn das Nutzerverhalten identifiziert werden kann. Es muss also ein System entworfen werden, das eine zuverlässige Vorhersage über das Mobilitätsverhalten des Nutzers ermöglicht.

1.1. Motivation

In dieser Arbeit wird untersucht, inwiefern sich das Nutzungsverhalten von Elektrofahrzeugen vorhersagen lässt, um potentiell den Energiefluss zwischen Fahrzeug und Stromnetz zu optimieren. Voraussetzung hierfür ist die Verfügbarkeit von V2G

(Vehicle-to-Grid), also der Fähigkeit von Elektrofahrzeugen, Energie nicht nur aufzunehmen, sondern auch wieder ans Stromnetz abzugeben. Obwohl V2G zunehmend an Präsenz gewinnt, bestehen in der praktischen Anwendung weiterhin große Lücken. Ein zentraler Baustein für das effiziente Zusammenwirken von Elektrofahrzeugen und Stromnetz ist ein zuverlässiges Vorhersagemodell des Mobilitätsverhaltens: Es erlaubt, Lade- und Entladezyklen so zu steuern, dass sowohl die Effizienz der mobilen Akkumulatoren als auch die Auslastung des Netzes optimiert werden. Entscheidend ist dabei die Zuverlässigkeit der Vorhersage, um ein fehlerloses System gewährleisten zu können.

1.2. Problemstellung

Die größte Hürde stellt eine stabile Datengrundlage dar. Da der Anwendungsfall sehr anspruchsvoll und dynamisch ist, lässt er sich mit bestehenden Ansätzen bislang nur unzureichend abbilden. Typischerweise erzielen Machine Learning Modelle gute Ergebnisse bei dieser Art von Vorhersageproblemen. Voraussetzung dafür ist allerdings eine hochqualitative Datenbasis, denn Fehleinschätzungen können zu einer fehlerhaften Energieverwaltung und im schlimmsten Fall zur Nichtverfügbarkeit des Fahrzeugs führen. Es müssen also verlässliche Daten über das Mobilitätsverhalten erhoben oder von Dritten bezogen werden. Da Qualität in diesem Bereich nicht genau definiert ist, gilt es dies an bestimmten Kriterien und Werten festzulegen.

In Betrachtung der existierenden wissenschaftlichen Literatur besteht im Moment ein Mangel an Studien, die sich explizit mit dem Mobilitätsverhalten von Individuen und der Erstellung von Prognosemodellen für diesen Zweck beschäftigen. Es gibt wertvolle Studien, die sich mit dem Mobilitätsverhalten von Gruppen beschäftigen, jedoch liefern diese wenig Aufschluss darüber, wie sich der Individualverkehr vorhersagen lässt. Damit künftig eine intelligente Steuerung von Energieflüssen möglich wird, müssen zuverlässige Vorhersagemodelle entwickelt werden, die auf hochqualitativen Daten zum individuellen Mobilitätsverhalten basieren.

Bisherige Arbeiten zur Prognose im Mobilitäts- und Energiekontext konzentrieren sich überwiegend auf den Ladezeitpunkt, die Leistung oder den letztendlichen Ladezustand sowie auf aggregiertes Nutzungsverhalten ganzer Gruppen. Das individuelle Fahr- und Nutzungsverhalten einzelner Fahrzeuge wird dabei kaum modelliert, obwohl gerade dieses für eine genaue Steuerung beziehungsweise Vorhersage von Nöten ist. Wo individuelles Fahrverhalten betrachtet wird, ist es zwischen den Fahrern stark heterogen, sodass ein einzelnes, übergreifendes Modell nicht den kompletten Kontext erfasst; zugleich ist die Datenbasis der wenigen einschlägigen Studien häufig

nicht ausreichend oder – wie bei synthetisch erzeugten Profilen – nicht zuverlässig repräsentativ für reale Mobilitätsdaten. Eine ausführliche Einordnung der relevanten Forschungsstränge und die daraus abgeleitete Forschungslücke finden sich in Abschnitt 2.4.

1.3. Zielsetzung

Das Ziel der Arbeit ist es, zuverlässige Modelle zur Prognose für individuelles Mobilitätsverhalten zu entwickeln. Die Modelle sollen den Zweck erfüllen, genauer Uhrzeiten vorhersagen zu können, wann ein Fahrzeug fährt und wann es parkt. Beispielsweise soll ein Modell zwischen ein und sieben Tage lange Prognosen erstellen können. Dazu sind die erhobenen Daten qualitativ zu bewerten und wenn nötig, aus Datenschutzgründen zu anonymisieren. Auf dieser Grundlage werden Modelle erstellt, die das Mobilitätsverhalten von Nutzern möglichst genau vorhersagen können. Des Weiteren soll Gegenstand der Arbeit sein, die Vorhersagegenauigkeit verschiedener Modelle zu vergleichen und deren Einflussfaktoren zu quantifizieren. Die Ergebnisse sollen dazu beitragen, eine intelligente Steuerung von Energieflüssen zwischen Fahrzeug und Stromnetz zu ermöglichen, um so die Nutzung von erneuerbaren Energien zu verbessern und eine wichtige Rolle bei der Energiewende zu spielen.

1.4. Forschungsfragen

Im Folgenden werden die Forschungsfragen der Arbeit vorgestellt, die in Tabelle 1.1 zusammengefasst sind. Die Forschungsfragen sind in zwei große Themenbereiche gegliedert: Mobilitätsprognose und Mobilitätsdaten. Der erste Bereich beschäftigt sich mit der Vorhersage des individuellen Mobilitätsverhaltens, während der zweite Bereich die Anforderungen an die Datengrundlage und deren Qualität untersucht.

Tabelle 1.1.: Forschungsfragen der Arbeit.

Nr.	Forschungsfrage
Mobilitätsprognose	
FF1	Wie zuverlässig lässt sich das individuelle Mobilitätsverhalten einzelner Fahrzeuge über einen Horizont von bis zu sieben Tagen vorhersagen?
FF1.1	Wie unterscheiden sich verschiedene Verfahren des maschinellen Lernens hinsichtlich ihrer Vorhersagegenauigkeit?
FF1.1.1	Welchen Einfluss hat die Datenqualität beziehungsweise -quelle auf die Prognosegüte – getrennt vom Einfluss des gewählten Algorithmus?
FF1.2	Welche Merkmale beeinflussen das individuelle Mobilitätsverhalten am stärksten?
Mobilitätsdaten	
FF2	Welche Anforderungen an die Datengrundlage ergeben sich daraus?
FF2.1	Wie wirken sich Aufbereitung und eine aus Datenschutzgründen notwendige Anonymisierung auf die Nutzbarkeit der Daten aus?
FF2.2	Wie lässt sich die Qualität der Daten bewerten?

1.5. Aufbau der Arbeit

Die vorliegende Arbeit ist wie folgt aufgebaut:

Kapitel 1 führt in das Thema ein und fasst den Kerninhalt zusammen.

Kapitel 2 erläutert die theoretischen Grundlagen, die für das Verständnis dieser Arbeit notwendig sind.

Kapitel 3 beschreibt die gewählte Methodik und das Vorgehen.

Kapitel 4 stellt die Implementierung der entwickelten Lösung vor.

Kapitel 5 präsentiert die Evaluation und die erzielten Ergebnisse.

Kapitel 6 diskutiert die Ergebnisse und deren Bedeutung.

Kapitel 7 fasst die Arbeit zusammen und gibt einen Ausblick auf zukünftige Arbeiten.

2. Theoretische Grundlagen

Dieses Kapitel legt die Grundlagen, auf denen die folgende Methodik aufbaut; Sowohl Begrifflichkeiten, als auch thematische Grundlagen werden erläutert. Als erstes wird festgelegt, was genau Mobilitätsdaten sind und wann sie relevant sind (Abschnitt 2.1). Darauf folgt der Blick darauf, wie Mobilitätsdaten verarbeitet werden und anhand welcher Kriterien sich ihre Qualität bewerten lässt (Abschnitt 2.2). Der vorletzte Punkt erläutert Machine Learning und wieso die Verwendung in dieser Arbeit unerlässlich ist. Als letztes ordnet Abschnitt 2.4 den aktuellen Stand der Forschung ein und arbeitet die Forschungslücke heraus, von der sich diese Arbeit in Abschnitt 2.4.1 abgrenzt.

2.1. Mobilitätsdaten

```
datetime,in_use,event,soc_pct,volt_v,curr_a,temp_c
2019-12-13 21:00:00,1,driving,58.8,412.8,-98.64,23.31
2019-12-13 22:00:00,1,driving,85.0,441.4,-47.771,27.52
2019-12-13 23:00:00,1,driving,86.0,436.76,37.899,28.5
2019-12-14 00:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 01:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 02:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 03:00:00,0,parked,86.0,436.76,37.899,28.5
2019-12-14 04:00:00,0,parked,86.0,436.76,37.899,28.5
```

(a) Datensatz: Cars Real World Electric (Pozzato u. a. 2023)

```
datetime,in_use,location,distance_km,consumption_kwh
2020-01-01 00:00:00,0,home,0.0,0.0
2020-01-01 01:00:00,0,home,0.0,0.0
2020-01-01 02:00:00,0,home,0.0,0.0
2020-01-01 03:00:00,0,home,0.0,0.0
2020-01-01 04:00:00,0,home,0.0,0.0
2020-01-01 05:00:00,0,home,0.0,0.0
2020-01-01 06:00:00,1,driving,15.0,3.377
```

(b) Datensatz: emobpy (Gaete-Morales u. a. 2020)

Abbildung 2.1.: Beispielhafte Auszüge zweier Mobilitätsdatensätze.

Wenn von Mobilitätsdaten gesprochen wird, ist im digitalen Kontext meist die Rede von Dateien, die Informationen in chronologischer Abfolge enthalten. Diese Daten

können in den unterschiedlichsten Formaten auftreten – mit beliebig vielen Einträgen und beliebig vielen Informationen pro Eintrag – und sind nicht zwingend homogen. Für das individuelle Fahr- und Nutzungsverhalten einzelner Fahrzeuge existiert – anders als für ÖPNV- oder Sharing-Daten – kein etabliertes Standardformat. Daraus folgt, dass sich Datensätze aus verschiedenen Studien erheblich in Schema und Granularität unterscheiden: In Abbildung 2.1a finden sich neben einem üblichen Zeitstempel nur die physikalischen Messwerte und kaum Informationen über das Fahrzeug oder den Zweck des Aufenthalts. Abbildung 2.1b enthält dagegen die Aktivität, gefahrene Distanz und den Energiekonsum. Für diese Arbeit sind jedoch nur wenige Felder unabdingbar – im Wesentlichen Zeitstempel und Fahr- beziehungsweise Parkzustand –, und diese sind in beiden Datensätzen enthalten. Trotz ihres erheblichen Unterschieds sind daher beide valide und relevant.

2.2. Verarbeitung von Mobilitätsdaten

Die Mobilitätsdaten bilden einen Hauptaspekt dieser Arbeit, da sie die Grundlage für das Training und die Validierung der Modelle darstellen.

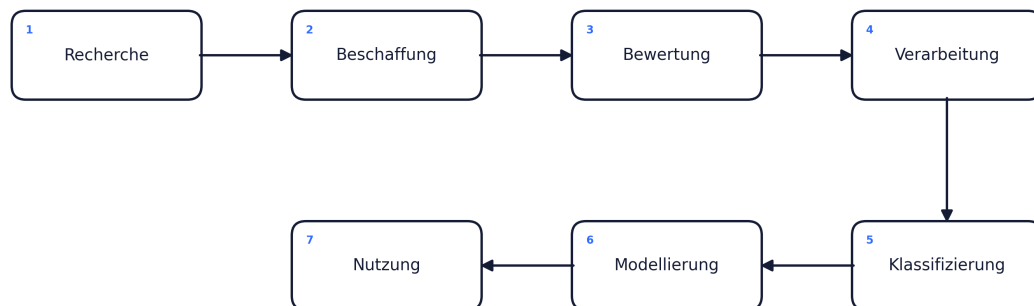


Abbildung 2.2.: Generalisierte Pipeline zur Verarbeitung von Mobilitätsdaten.

Abbildung 2.2 zeigt den Ablauf, den die Daten von der Recherche bis zur Nutzung durchlaufen; jeder der sechs Abschnitte umfasst dabei mehrere Teilschritte. Tabelle 2.1 fasst diese je Phase zusammen, die folgenden Absätze erläutern sie.

Phase	Wesentliche Teilschritte
1 – Recherche	Identifikation relevanter Studien und Datensätze; Prüfung der thematischen Eignung.
2 – Beschaffung	Bezug der Daten unter Beachtung rechtlicher und datenschutzrechtlicher Vorgaben.
3 – Bewertung	Beurteilung der Datenqualität anhand definierter Kriterien.
4 – Verarbeitung	Bereinigung, Überführung in ein einheitliches Schema, Feature-Engineering und gegebenenfalls Anonymisierung.
5 – Modellierung	Training und Validierung der Klassifikations- und Prognosemodelle.
6 – Nutzung	Anwendung der Modelle zur Vorhersage und Bereitstellung der Ergebnisse für nachfolgende Systeme.

Tabelle 2.1.: Phasen der Mobilitätsdaten-Pipeline

Recherche: Der erste Schritt ist die Identifikation geeigneter Datenquellen. Dabei ist zu beachten, dass Mobilitätsdaten allein noch keine Relevanz bedeuten: Die Daten müssen einer Studie entstammen, die für die individuelle Fahrzeugnutzungsprognose einschlägig ist. Erst nach hinreichender Prüfung dieser thematischen Eignung werden die Daten weiterverfolgt.

Beschaffung: Geeignete Daten müssen anschließend bezogen werden. Durch herunterladen offener Datensätze, Anfragen bei datenhaltenden Dritten oder eigene Erhebung. Bereits hier sind rechtliche und datenschutzrechtliche Rahmenbedingungen zu berücksichtigen, da personenbezogene Bewegungsdaten besonders schützenswert sind. Da jedoch der zeitliche Rahmen der Arbeit nicht für eine eigene Erhebung ausreicht, sind die Daten auf öffentlich zugängliche Daten oder Kooperationen beschränkt.

Bewertung: Vor der Weiterverarbeitung wird die Qualität der Daten beurteilt. Da der Begriff der Datenqualität in diesem Bereich nicht eindeutig definiert ist, wird er an konkreten Kriterien festgemacht: Vollständigkeit, zeitliche Auflösung und Umfangänglichkeit der Daten. Diese Bewertung entscheidet letztendlich darüber, ob ein Datensatz für ein zuverlässiges Modell geeignet ist.

Verarbeitung: Die als geeignet bewerteten Daten werden bereinigt und in ein einheitliches Schema überführt, sodass Quellen unterschiedlicher Herkunft vergleichbar werden. Anschließend werden aus den Rohdaten die für die Modelle benötigten Merkmale abgeleitet und bei Bedarf erfolgt zudem eine Anonymisierung.

Modellierung: Auf der aufbereiteten Datenbasis werden die Modelle trainiert und validiert. Dieser Schritt umfasst sowohl die Klassifikation des Fahrverhaltens als auch die eigentliche Prognose der Fahrzeugnutzung.

Nutzung: Abschließend werden die trainierten Modelle angewendet, um das individuelle Mobilitätsverhalten vorherzusagen. Die daraus gewonnenen Prognosen werden dokumentiert und bereitgestellt. Sie liefern die Grundlage für Vorhersagen in der Fahrzeugnutzung und damit einen weiteren Schritt hin zu einem intelligenten Energiesystem.

2.3. Machine Learning

Das Prinzip des maschinellen Lernens besteht darin, einem Computer ein gewünschtes Verhalten anhand von Beispieldaten beizubringen, anstatt es durch einen fest vorgegebenen, regelbasierten Algorithmus explizit zu programmieren (Bishop 2006). Das Modell erkennt dabei statistische Muster in den Trainingsdaten und nutzt diese, um Vorhersagen für bislang ungesehene Eingaben zu treffen. Der Vorteil gegenüber einem starren Algorithmus liegt darin, dass komplexe Zusammenhänge – wie das individuelle Mobilitätsverhalten – erlernt werden können, ohne dass sie vorab vollständig bekannt sein müssen.

Häufig wird maschinelles Lernen mit künstlichen neuronalen Netzen gleichgesetzt, deren Aufbau vom menschlichen Gehirn inspiriert ist. Neuronale Netze bilden jedoch nur eine von vielen Verfahrensklassen. Die in dieser Arbeit eingesetzten Modelle – Random Forest und Gradient Boosting (Abschnitt 2.3.8 und Abschnitt 2.3.9) – beruhen stattdessen auf Entscheidungsbäumen und sind nicht biologisch motiviert.

2.3.1. Überwachtes Lernen

In dieser Arbeit kommt ausschließlich überwachtes Lernen zum Einsatz (Hastie u. a. 2009). Dabei lernt das Modell aus Beispielpaaren von Eingaben und zugehörigen,

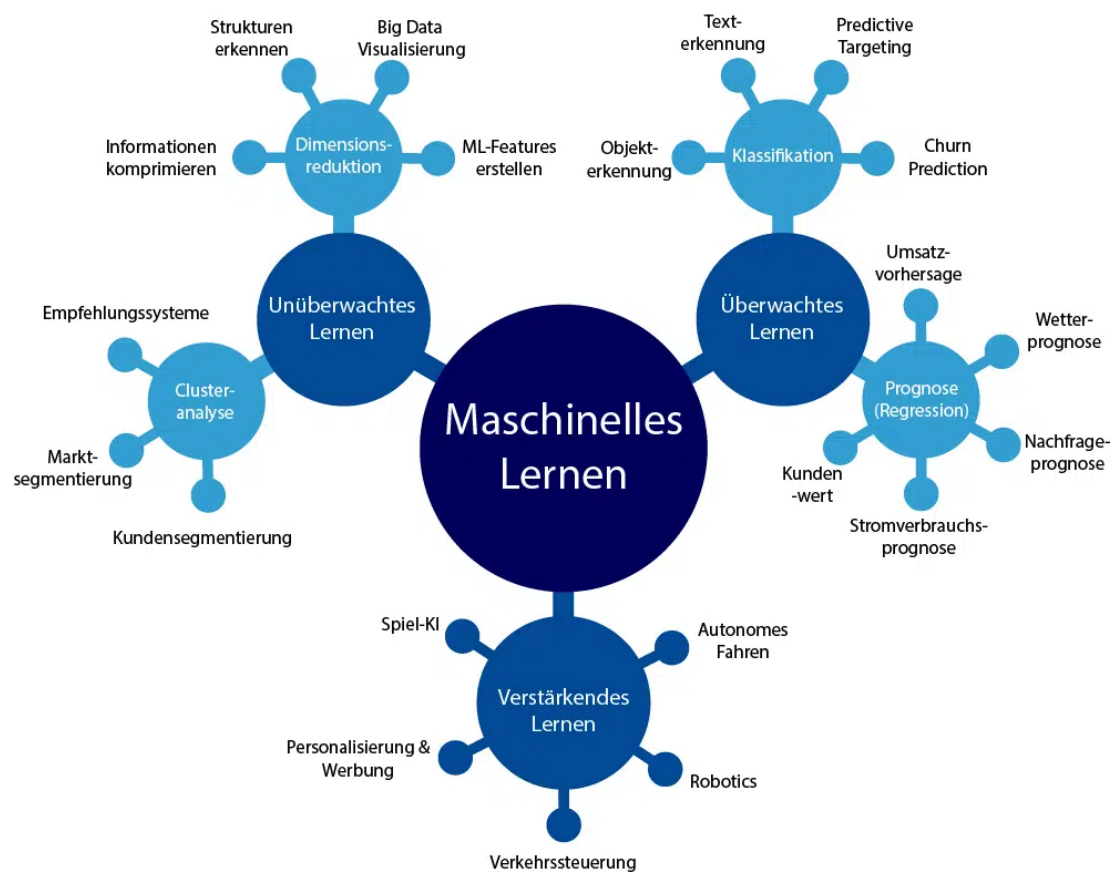


Abbildung 2.3.: Teilbereiche des maschinellen Lernens mit beispielhaften Anwendungen (Wuttke 2024).

bekannten Zielwerten. Je nach Art des Zielwerts werden zwei Aufgabentypen voneinander unterschieden, die in dieser Arbeit Anwendung finden: Klassifikation, bei der eine diskrete Klasse vorhergesagt wird (etwa ob ein Fahrzeug in einer gegebenen Stunde fährt oder geparkt ist) und Regression, bei der ein kontinuierlicher Wert geschätzt wird (etwa die Abfahrts- oder Rückkehrzeit).

2.3.2. Merkmale und Feature Engineering

Bei Machine Learning ist oft die Rede von Feature Engineering oder das Umformen von Merkmalen (Géron 2022). Ein Merkmal ist beispielsweise eine Spalte in einer Comma-Separated-Value Datei (siehe Abbildung 2.1), hierbei könnte es sich um jede der Spalten handeln. Folglich ist der Zeitstempel, die Handlung, der Ort oder auch der Energiekonsum ein Merkmal beziehungsweise Feature.

In der Regel sind die Daten in ihrer Rohform nicht direkt aussagekräftig, weswegen sie interpretiert werden müssen. Ein Zeitstempel (Beispiel: 2026-05-25 08:00:00) hat ohne Kontext keinen bestimmten Wert, daher werden durch Feature Engineering die verborgenen Informationen extrahiert und macht sie für das Modell zugänglich. Der genannte Zeitstempel beispielsweise ist im Kontext von Mobilität:

- Ein Montag → Werktag
- Acht Uhr → Pendlerzeit

Ein Modell entdeckt Muster nicht von selbst, daher müssen die Informationen verwertbar gemacht werden. Tage die nicht auf einen Werktag fallen, oder eine andere Uhrzeit haben, die keine Pendlerzeit ist, können je nach restlichem Informationsgehalt (Ort, Fahrtdistanz, Aufenthaltsdauer) zu einem jeweils anderen Ergebnis führen.

Lag-Merkmal

Ein Lag ist der Wert eines Merkmals aus einem früheren Zeitschritt (Hyndman u. a. 2021). Lags sind teilweise periodisch und mit Stundenwerten gleichzusetzen. Ein **Lag_1** beispielsweise ist das selbe Event, nur eine Stunde in der Vergangenheit, selbiges Schema gilt für einen **Lag_168**, dieser liegt 168 Stunden in der Vergangenheit. Es gibt sehr spezifische Werte und diese erlauben auf bestimmte Muster Rückschlüsse. Lags von eins, zwei, 24 und 168, wobei 24 Stunden einen Tag und 168 Stunden sieben Tage in der Vergangenheit abbilden.

Ein Risiko, das durch Lag auftreten kann, ist die Rechnung über mehr als ein Fahrzeug hinweg. Dadurch kann ein Verhalten auf ein anderes Fahrzeug interpretiert werden und falsch zugewiesene Verhaltensmuster errechnen.

Rolling-Aggregate

Ein „Rolling Mean“ oder Rolling-Mittel, fasst ein Zeitfenster zu einer Kennzahl zusammen (Hyndman u. a. 2021). So werden beispielsweise die letzten 24 Stunden oder sieben Tage Aktivitätsanteil zusammengefasst.

Data Leakage

Data Leakage ist ein Risiko, das bei einem Rolling Mean auftreten kann, indem es die Zukunft einbezieht oder sogar den Zielwert selbst mit einrechnet (Kaufman u. a. 2012). Dadurch lernt das Modell den tatsächlich gewünschten Wert und kann diesen, fälschlicherweise, richtig wiedergeben. Eine Falschimplementierung kann dazu führen, dass die Tests übermäßig gut abschneiden, im Realfall jedoch komplett versagen.

2.3.3. Training

Wenn ein Modell trainiert wird, passt es die eigenen internen Parameter an die Trainingsdaten an, sodass Vorhersagen möglichst gut zu den gelernten Zielwerten passen (Bishop 2006). Formal wird dabei das Resultat einer Verlustfunktion minimiert, welche misst, wie weit die Vorhersage von der Wahrheit abweicht. Bei einem baumbasierten Modell werden konkret Splits gewählt für bestimmte Merkmale und Schwellen, die gewisse Zielgrößen im jeweiligen Knoten am besten trennen (siehe Abschnitt 2.3.7).

2.3.4. Generalisierung

Das Prinzip der Generalisierung beschreibt, ein Modell so zu trainieren, dass es die Muster der Daten erkennt und deren Berechnung erlernt (Hastie u. a. 2009). Das bedeutet, dass das Modell auch mit neuen Daten – die ungesehene Inhalte verwenden – umgehen kann, die nah am gelernten Schema liegen. Tabelle 2.2 fasst zusammen, wie sich Unteranpassung, gute Anpassung und Überanpassung anhand von Trainings- und Testfehler unterscheiden lassen.

Overfitting

Bei einem überangepassten Modell sind Trainingsfehler niedrig und Testfehler hoch. Kurzum bedeutet das, dass das Modell die Daten zu gut kennt und diese quasi auswendig weiß und dieses Wissen und Voreingenommenheit auf neue Daten fälschlicherweise anwendet.

Underfitting

Ein Modell das unterangepasst ist, schneidet weder beim Training noch beim Test gut ab. Hier ist das Modell zu simpel beziehungsweise seine Kapazität zu gering, um das zugrunde liegende Muster zu erfassen. Fehler bleiben dadurch bei beiden Datensätzen hoch.

Anpassung	Trainingsfehler	Testfehler	Charakteristik
Underfitting	hoch	hoch	Modell zu simpel; das zugrunde liegende Muster wird nicht erfasst.
Gute Anpassung	niedrig	niedrig	Kleine Generalisierungslücke; die Muster werden auf neue Daten übertragen.
Overfitting	sehr niedrig	hoch	Große Generalisierungslücke; Trainingsdaten werden inklusive Rauschen auswendig gelernt.

Tabelle 2.2.: Charakterisierung von Anpassungen anhand von Fehlerwerten Géron 2022.

2.3.5. Validierung

Durch die Validierung ist es möglich der Genauigkeit des Modells einen numerischen Wert zuzuweisen, was funktioniert, indem das Modell Daten bewertet, die bisher nicht bekannt waren.

$$\text{CV}(\hat{f}) = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{f}^{-\kappa(i)}(x_i)) \quad (2.1)$$

Die Gleichung (2.1) beschreibt die Genauigkeit eines Modells auf einer bestimmten Menge Daten. Hierfür wird ein Datensatz in K gleichgroße Mengen unterteilt – auch *Folds* genannt – gefolgt von K trainings. Da K *Folds* existieren, wird das Modell einmal mit jedem *Fold* getestet und hat die restlichen *Folds* zur Grundlage.

2.3.6. Bewertungsmetriken

Die Güte des Modells wird durch die Bewertungsmetriken bestimmt, die sich je nach Art der Zielgröße unterscheiden. Bei Klassifikationsproblemen werden Metriken wie Genauigkeit, Präzision, Recall und F1-Score verwendet, bei Regressionsproblemen hingegen Metriken wie mittlerer absoluter Fehler (MAE), mittlerer quadratischer Fehler (MSE) und R^2 . Abhängig von der Art des Problems und den Zielen der Analyse werden die entsprechenden Metriken ausgewählt, um die Leistung des Modells entsprechend zu bewerten.

2.3.7. Entscheidungsbäume

Entscheidungsbäume bilden einen Teil des maschinellen Lernens, sind jedoch grundlegend verschieden zu den menschlich inspirierten neuronalen Netzen. Entscheidungsbäume imitieren ein regelbasiertes und hierarchisches Vorgehen. Getrieben von Daten entscheidet deren Label-Vollständigkeit, ob ein Baum supervised oder unsupervised trainiert wird. So sorgen ungelabelte Daten dafür, dass ein Baum unsupervised trainiert wird. Das bedeutet, dass der Baum selbstständig Muster in den Daten erkennt und diese in einer Hierarchie abbildet. Bei einem (semi-)supervised Baum hingegen, werden die Daten entweder vollständig oder teilweise mit Labels versehen und der Baum lernt anhand dieser Labels die Abbildung auf die Zielwerte. Unabhängig davon entscheidet der Typ der Zielvariable, ob ein Baum regressiv oder klassifikatorisch trainiert wird.

Gini Index

Die allgemeine Form der Gini-Unreinheit für einen Knoten mit C Klassen und den Klassenanteilen p_i lautet (Breiman u. a. 1984):

$$G = 1 - \sum_{i=1}^C p_i^2 \quad (2.2)$$

Für den binären Fall mit zwei Klassen ($p_1 = p$, $p_0 = 1 - p$) vereinfacht sich Gleichung (2.2) zu (Breiman u. a. 1984):

$$G = 1 - (p^2 + (1 - p)^2) = 2p(1 - p) \quad (2.3)$$

Hier ist die Rede von Unreinheit: einer Metrik, die ausdrückt, mit welcher Wahrscheinlichkeit ein Wert einer falschen Klasse zugeordnet würde, wenn er anhand der

Klassenverteilung im Knoten ein zufälliges Label erhielte. Die gewichtete Unreinheit eines Splits ergibt sich aus den Unreinheiten der beiden Kindknoten (Breiman u. a. 1984):

$$G_{\text{split}} = \frac{n_{\text{links}}}{n} G_{\text{links}} + \frac{n_{\text{rechts}}}{n} G_{\text{rechts}} \quad (2.4)$$

In Gleichung (2.4) zeigt n die Elemente im Elternknoten und G die Unreinheit des jeweiligen Knoten an. Der Rest funktioniert analog, anhand der vorausgegangenen Formeln. Mit der Split-Formel lassen sich mehrere Iterationen pro Knoten vornehmen, wovon dann der reinste Split verwendet wird, um im Baum eine Rekursionsebene tiefer zu schreiten. Insgesamt funktioniert dieser Split $n - 1$ mal.

Abbildung 2.4 zeigt einen beispielhaften Entscheidungsbaum, welcher je Knoten berechnet Gini-Wert, Stichprobengröße und Klassenverteilung besitzt.

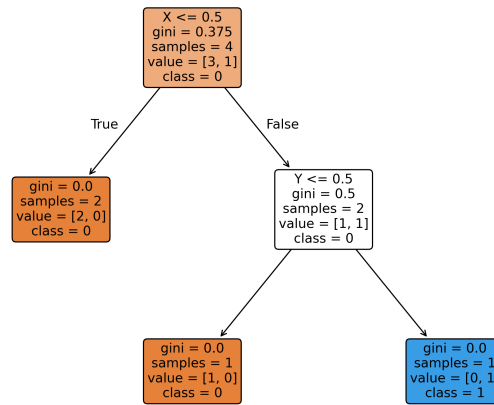


Abbildung 2.4.: Entscheidungsbaum mit Aufteilungskriterien (nach Schilli 2017).

Bagging und Boosting

Das Baggingverfahren sorgt dafür, dass die Bäume von einer in der realen Trainingsmenge existierenden Untermenge lernen. Es werden aus der Ursprungsmenge N Stichproben gezogen, auf denen jeweils ein Classifier/Regressor trainiert wird (Abbildung 2.5, ResearchGate 2022). Für jede Stichprobe wird ein Modell trainiert, das anschließend zu einem Ensemble aggregiert wird. Dadurch, dass alle Bäume auf unterschiedlichen Stichproben trainiert werden, wird die Varianz des Modells reduziert und die Vorhersagegenauigkeit verbessert. Bagging ist besonders effektiv bei Modellen, die zu Überanpassung neigen.

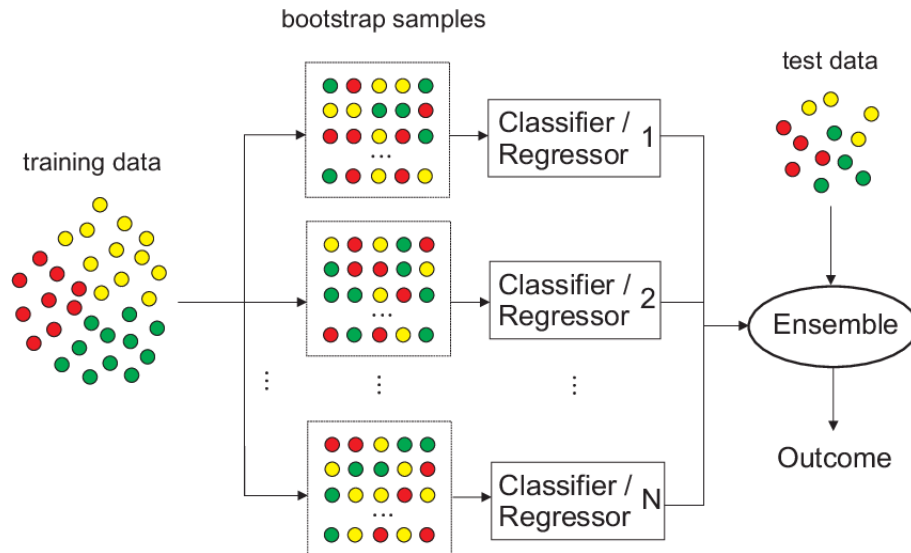


Abbildung 2.5.: Bagging: Aus den Trainingsdaten werden N Bootstrap-Stichproben gezogen, auf denen jeweils ein Baum trainiert wird; deren Vorhersagen werden zu einem Ensemble aggregiert (ResearchGate 2022).

Boosting sorgt dafür, dass die Bäume auf den Residualfehlern der vorherigen Iteration trainiert werden (Abbildung 2.6, ResearchGate 2021). Dadurch wird die Vorhersagegenauigkeit des Modells verbessert. Im Gegensatz zu Bagging, bei dem die Bäume unabhängig voneinander trainiert werden, werden beim Boosting die Bäume sequentiell trainiert, wobei jeder Baum auf den Fehler des vorherigen Baums aufbaut. Dies ermöglicht es dem Modell, sich auf schwierige Beispiele zu konzentrieren und die Gesamtleistung zu verbessern. Im Kern besteht Boosting darin, schwache Lernalgorithmen zu kombinieren, um einen starken Lernalgorithmus durch gewichtete Aggregation zu erstellen. Boosting ist besonders effektiv bei Modellen, die zu Unteranpassung neigen.

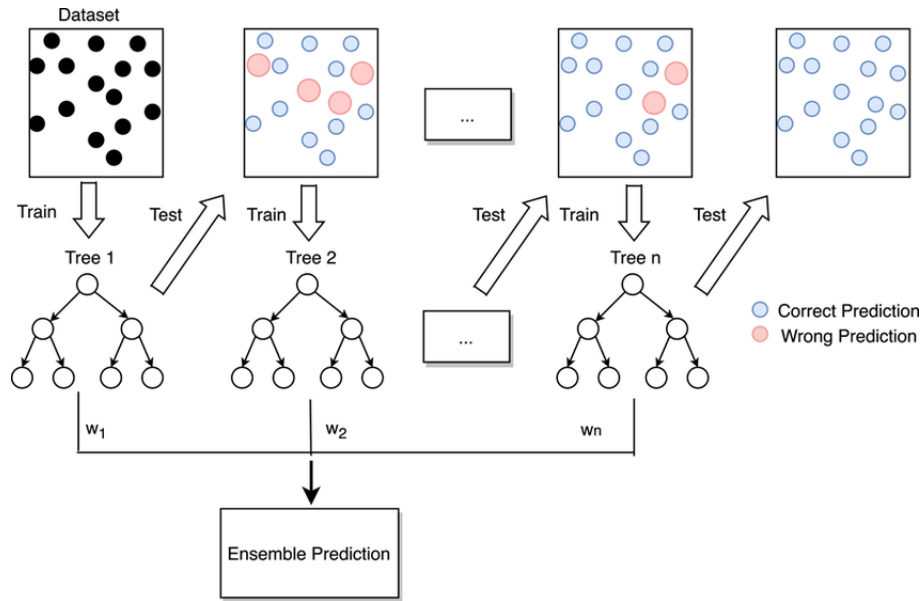


Abbildung 2.6.: Boosting: Die Modelle werden nacheinander trainiert, wobei jedes folgende Modell auf den Fehlern des vorherigen Modells basiert (ResearchGate 2021).

In der nachfolgenden Sektion werden der Random Forest in Abschnitt 2.3.8 und die Light Gradient Boosting Machine in Abschnitt 2.3.9 vorgestellt, die auf den zuvor beschriebenen Prinzipien von Entscheidungsbäumen, Bagging und Boosting basieren. Abbildung 2.7 veranschaulicht den zentralen Unterschied im Baumwachstum: LightGBM wächst *leaf-wise* so, dass Knoten mit dem größten Fehlerbeitrag aufgeteilt werden, während klassische Verfahren, wie der Random Forest, *level-wise* wachsen.

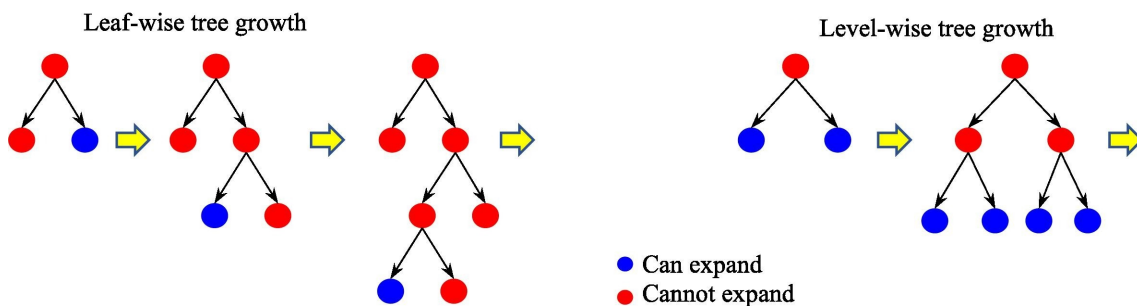


Abbildung 2.7.: Baumwachstum in Ensemble-Verfahren (ScienceDirect 2026).

2.3.8. Random Forest

Ein Typ der Entscheidungsbaum-Modelle ist der Random Forest. Dieser besteht im Normalfall aus mehreren Entscheidungsbäumen, die, bei Bagging, jeweils auf einem zufälligen Teil der Trainingsdaten trainiert werden (Abbildung 2.5). Die Vorhersagen einzelner Bäume werden anschließend zu einem Ensemble aggregiert,

um die Gesamtvorhersage zu verbessern. Random Forest ist besonders effektiv bei Modellen, die zu Überanpassung neigen, da die Aggregation der Vorhersagen die Varianz reduziert und die Generalisierungsfähigkeit verbessert.

2.3.9. Light Gradient Boosting Machine

Die Light Gradient Boosting Machine (LightGBM) ist ein Gradient-Boosting-Mechanismus, der auf Entscheidungsbäumen basiert, welche auf den Residualfehlern der vorherigen Iteration trainiert werden. In dieser Arbeit wird LightGBM ohne Bagging verwendet aber mit Boosting, um die Vorhersagegenauigkeit zu verbessern. LightGBM ist besonders effektiv bei Modellen, die zu Unteranpassung neigen, da es die Vorhersagegenauigkeit verbessert, indem es sich auf die Fehler der vorherigen Iteration konzentriert und *leaf-wise* (Abbildung 2.7) wächst.

2.3.10. Unsicherheit und Quantilsregression

Es gibt zwei Arten der Unsicherheit, die bei Prognosen betrachtet werden müssen:

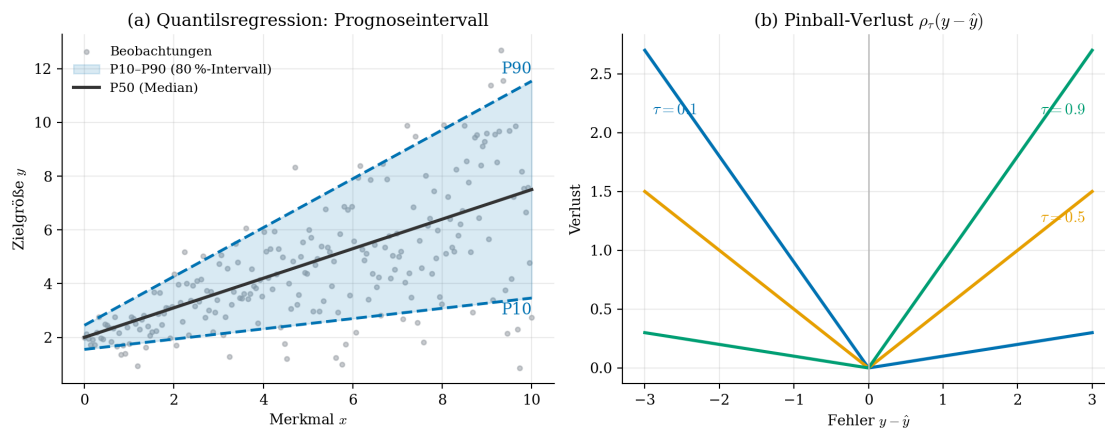
- **Aleatorische Unsicherheit:** Hier wird der Zufall im Prozess selbst betrachtet, der nicht durch weitere oder bessere Daten reduziert werden kann. Beispielsweise kann das Wetter nicht deterministisch vorhergesagt werden, da es intrinsisch zufällig ist. Unter anderem wird diese auch als Datenunsicherheit bezeichnet.
- **Epistemische Unsicherheit:** Hier wird die Unsicherheit betrachtet, die durch unvollständige Informationen oder unzureichendes Wissen über den Prozess entsteht. Diese Unsicherheit kann durch zusätzliche Daten oder bessere Modelle reduziert werden. Unter anderem wird diese auch als Modellunsicherheit bezeichnet.

Eine Punktprognose ist für eine Unsicherheit nicht ausreichend, da sie nur einen Wert liefert und keine Informationen über die Streuung oder die Verteilung liefert. Während die klassische Regression den bedingten Mittelwert schätzt, sagt die Quantilsregression gezielt einzelne Quantile Q_τ der Zielverteilung vorher – im Normalfall das 10., 50. und 90. Perzentil (Koenker u. a. 1978). Zur Erreichung wird der asymmetrische Pinball-Verlust minimiert:

$$\rho_\tau(y - \hat{y}) = \max(\tau(y - \hat{y}), (\tau - 1)(y - \hat{y})) \quad (2.5)$$

Die Formel 2.5 ist definiert durch:

- y ist der tatsächlich beobachtete Wert
- $\hat{y}$ ist der vorhergesagte Wert
- τ ist das Quantil, das geschätzt werden soll, z. B. 0,1, 0,5, 0,9
- ρ_τ ist Strafe oder der Pinball-Verlust, der die Abweichung zwischen dem beobachteten Wert und dem vorhergesagten Quantil misst



Abbildungung 2.8.: (a) Heteroskedastisches Prognoseband: Median (P50) und 80 %-Intervall (P10–P90), dessen Breite mit der Streuung wächst. (b) ρ_τ für $\tau = 0,1/0,5/0,9$; die Asymmetrie schiebt die Schätzung an das jeweilige Quantil (Koenker u. a. 1978).

Beim Pinball-Verlust werden Unter- und Überschätzung je nach τ unterschiedlich gewichtet. Das Intervall zwischen $Q_{0,1}$ und $Q_{0,9}$ bildet somit ein 80 %-Prognoseband (Abbildungung 2.8). Quantile werden in dieser Arbeit auf drei Wegen erzeugt: direkt per Pinball-Verlust im LightGBM-Forecaster, über die Streuung der Bäume im Random Forest (Meinshausen 2006) sowie über die Pfad-Streuung des Monte-Carlo-Rollouts.

2.3.11. Long Short-Term Memory

Das LSTM ist ein rekurrentes neuronales Netz (RNN) mit einem Zellzustand und drei Gattern (Forget, Input, Output), die den Zustand der Zelle beeinflussen. Ein LSTM ist dazu entwickelt, Langzeitabhängigkeiten in Sequenzen zu lernen und ist insbesondere geeignet für Zeitreihenprognosen. Durch die Gatter kann das LSTM entscheiden, welche Informationen beibehalten werden und welche verworfen werden, wodurch es in der Lage ist, relevante Muster über längere Zeiträume hinweg zu erfassen. Das LSTM umgeht das Vanishing-Gradient-Problem, das bei traditionellen RNNs auftritt, indem es den Zellzustand über viele Zeitschritte hinweg beibehält und so die Weitergabe von Informationen über lange Sequenzen ermöglicht.

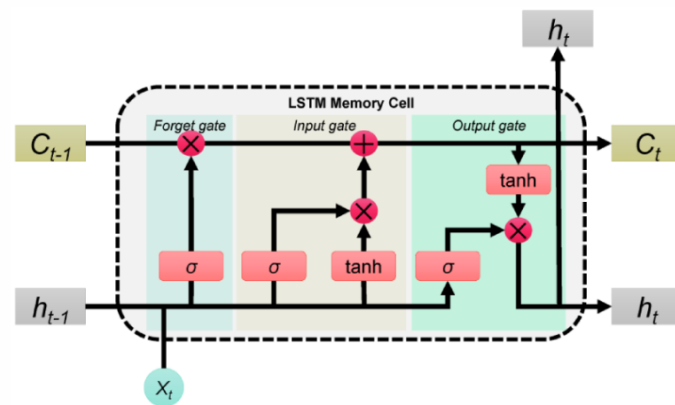


Abbildung 2.9.: Aufbau einer LSTM-Zelle: Der Zellzustand C_t wird durch Forget-, Input- und Output-Gate gesteuert, die per σ - und $\tanh$ -Aktivierung festlegen, was aus dem Gedächtnis verworfen, neu geschrieben und ausgelesen wird (dida 2026).

Die Abbildung 2.9 stellt den Aufbau einer LSTM-Zelle dar, die aus dem Zellzustand C_t , drei Gattern (Forget, Input, Output) und den Aktivierungsfunktionen σ und $\tanh$ besteht. C_t , der Zellzustand, ist für die Speicherung von Informationen verantwortlich, während die Gatter bestimmen, welche Informationen beibehalten, verworfen oder neu geschrieben werden. Die Aktivierungsfunktionen σ und $\tanh$ steuern, wie die Informationen im Zellzustand verarbeitet werden und ermöglichen es dem LSTM auf diese Weise, relevante Muster in den Eingabesequenzen zu erkennen und somit Abhängigkeiten über längere Zeiträume zu erkennen. Zeitreihenprognosen, Sprachverarbeitung und Anomalieerkennung sind typische Aufgaben, bei denen LSTMs erfolgreich eingesetzt werden, da sie durch ihr Gedächtnis in der Lage sind, komplexe zeitliche Abhängigkeiten zu modellieren und Vorhersagen auf Basis historischer Daten zu treffen.

2.4. Stand der Forschung

Die Forschung zur Prognose individueller Fahrzeugnutzung lässt sich grob in mehrere Sektionen gliedern, die sich in Zielgröße, Methodik und Datengrundlage unterscheiden. Tabelle 2.3 fasst diese Sektionen als Übersicht zusammen. Jede dieser Sektionen beinhaltet Arbeiten, die zum Teil entweder relevant für diese Arbeit, oder zu Ihrer Eingrenzung dient. Die folgenden Unterabschnitte vertiefen die Sektionen jeweils und arbeiten am Ende die für diese Arbeit zentrale Forschungslücke heraus.

Tabelle 2.3.: Zentrale Forschungssektionen im Umfeld der individuellen Fahrzeugnutzungsprognose.

Forschungsstrang	Typische Methoden	Datengrundlage	Offene Punkte
Mobilitäts- und Nutzungsfore-cast	Statistische Modelle (ARIMA), Random Forest, neuronale Netze (LSTM)	Reale Telematik- und Bewegungsdaten	Generalisierung über Fahrzeuge und Quellen hinweg
Driving-/Parking-Klassifikation	Random Forest, SVM, Gradient Boosting	EV-Flottendaten, stündliche Profile	Einfluss der Datenqualität auf die Klassifikationsgüte
EV-Last- und Ladeprognose	Gradient Boosting, Deep Learning	Synthetische Daten (z. B. emobpy), reale EV-Daten	Vergleichbarkeit synthetischer und realer Datengrundlagen
Unsicherheits- und Quantilsprognose	Quantilsregression, Quantile-Random-Forest	Heterogene Mobilitätsdaten	Kalibrierung individueller Vorhersageintervalle (P10/P90)

2.4.1. Verwandte Arbeiten

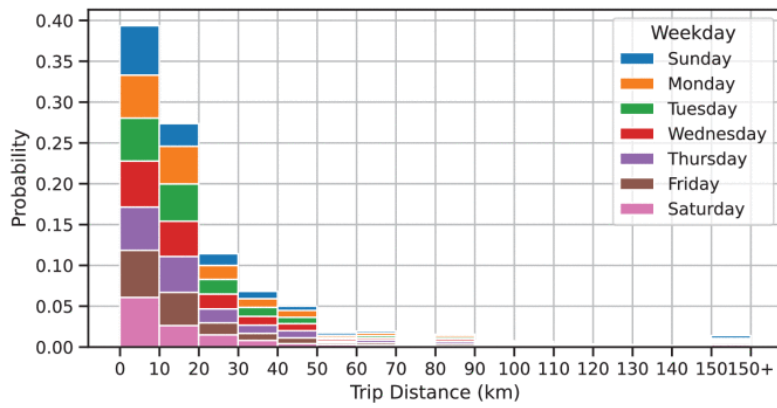
Um zu verdeutlichen, womit sich diese Arbeit im wesentlichen auseinandersetzt, werden im Folgenden verwandte Arbeiten vorgestellt und voneinander abgegrenzt. Die Auswahl der Arbeiten orientiert sich an den in Tabelle 2.3 dargestellten Forschungssträngen. Dabei werden sowohl die Zielsetzung, als auch die Methodik der jeweiligen Arbeiten betrachtet, um die Unterschiede und Gemeinsamkeiten herauszuarbeiten.

Mobilitäts- und Nutzungsforecasts

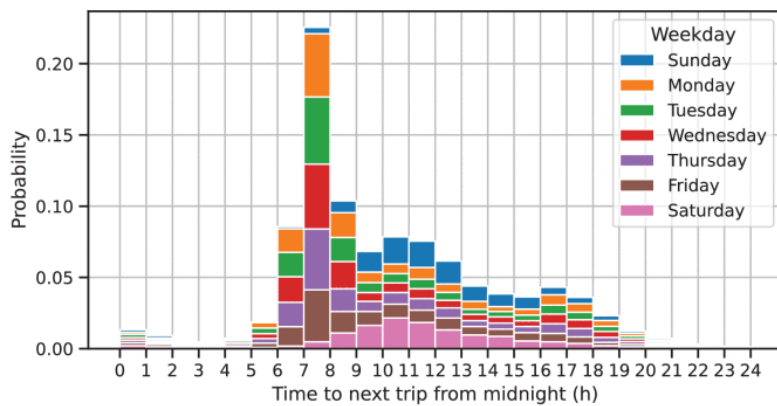
Klassische Zeitreihenmodelle wie ARIMA, aber auch moderne Ansätze wie Random Forest und neuronale Netze, insbesondere LSTM sind in Verwendung, um für Mobilitätsverhalten Prognosen zu erstellen. Die Daten bestehen zum Großteil aus realen Telematik- und Bewegungsdaten, die eine hohe Granularität aufweisen. Ein zentrales Ziel hierbei ist die Generalisierung über eins oder mehrere Fahrzeuge hinweg, um Vorhersagen für zukünftiges Fahrverhalten treffen zu können. Die Forschungsarbeiten

haben den Zweck, die Genauigkeit der Vorhersagen zu maximieren und gleichzeitig die Interpretierbarkeit der Modelle zu gewährleisten.

Eine Studie beispielsweise befasst sich mit der Vorhersage von Fahrverhalten auf Grund von temperaturbedingter Batterieeffizienz – denn besonders niedrige Temperaturen können die Effizienz der Batterien erheblich beeinflussen (Lindroth u. a. 2024). Es werden Online-Lernmodelle verwendet, um die Abfahrtszeit und die Distanz für den ersten Trip des Tages vorherzusagen. Abbildung 2.10 zeigt die zugrunde liegenden Verteilungen von Fahrdistanz und Abfahrtszeit, aufgeschlüsselt nach Wochentag. Es ist gut zu erkennen, dass die Fahrdistanz – unabhängig vom Wochentag – eine ähnliche Verteilung aufweist, während die Abfahrtszeit stark vom Wochentag abhängt. Eine weitere Studie entwickelt ein LSTM-Modell mit raum-zeitlichem Bewertungsmechanismus, das die individuelle Fahrzeugnutzung vorhersagt. Reiseziele, aktuelle Fahrtdaten (Echtzeit-Bewegungen) und historische Bewegungsmuster werden hierbei als Datengrundlage verwendet (others 2025). Die Ergebnisse zeigen, dass das Modell im Vergleich zu spezifischen Random Forest, Distant Neighboring Dependencies, Location Semantics and Location Importance -LSTM und Intersection Transfer Preference and Current Movement Mode um ca. 10-15% Genauigkeit übertrifft.



(a) Driving distance



(b) Departure time

Abbildung 2.10.: Verteilung der Fahrdistanz (a) und der Abfahrtszeit des ersten Tagestrips (b) nach Wochentag (Lindroth u. a. 2024).

Driving-/Parking-Klassifikation

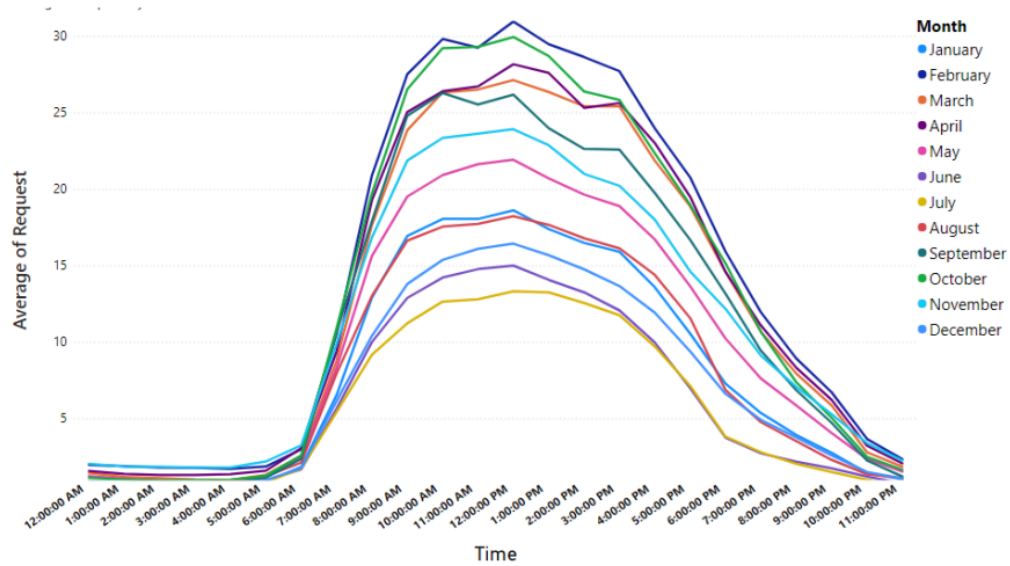
Die Identifikation von Fahr- und Parkzuständen ist ein zentraler Aspekt der individuellen Fahrzeugnutzungsprognose, daher ist es wichtig diese beiden Klassen zuverlässig zu unterscheiden. In der Literatur werden hierfür unterschiedliche Methoden eingesetzt. Eine Studie versucht gezielt Fahrer in bestimmte Klassen zu unterteilen, um deren Fahrverhalten besser zu verstehen (Kumar u. a. 2023). In der Studie werden IoT und Machine Learning verwendet, um Fahrverhalten anhand von OBD-II-Daten (beispielsweise Geschwindigkeit, Motordrehzahl, Bremsmuster) ohne zusätzliche Sensoren zu analysieren. Mithilfe von Support Vector Machines, AdaBoost und Random Forest werden die Fahrer in zehn Klassen unterteilt, die sich in Kraftstoffverbrauch, Lenkstabilität, Geschwindigkeitsstabilität und Bremsverhalten unterscheiden. Eine weitere Studie untersucht die Klassifikation von Fahr- und Parkzuständen, um das Verhalten präziser zu verstehen und die Genauigkeit der Vorhersagen zu verbessern (Su u. a. 2025). Die Methodik beinhaltet ein Clustering per DBSCAN, welches häufig

besuchte Orte identifiziert und eine Ableitung von Gebäudekategorien vornimmt.

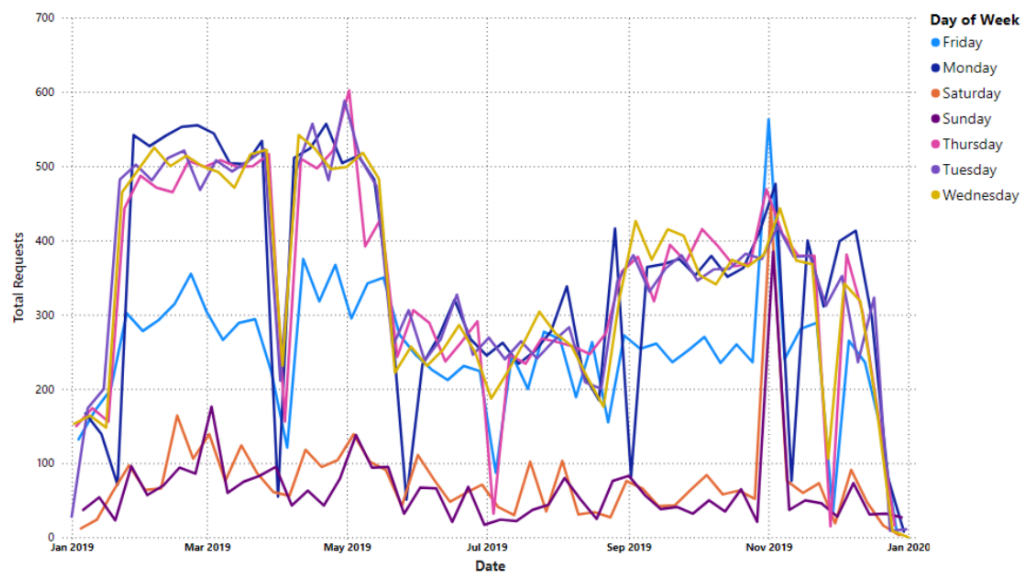
EV-Last- und Ladeprognose

Im Bereich der EV-Last- und Ladeprognose werden vielfältig Methoden angewandt, um die Ladebedarfe und Lastprofile von Elektrofahrzeugen zu prognostizieren. Eine Studie verfolgt hierbei einen probabilistischen Ansatz und quantifiziert die Unsicherheit des Ladeverhaltens mittels Modellen wie GAMLSS oder QGAM, um die Ladebedarfe als vollständige Prognoseintervalle statt als reine Punktwerte vorherzusagen (Amara-Ouali u. a. 2025). Eine weitere Studie entwickelt ein multivariates long short-term memory (LSTM) Modell mit einem Attention-Mechanismus, um den Ladebedarf in 15-Minuten-Intervallen vorherzusagen (Sanami u. a. 2025). Des Weiteren erhält das Modell zusätzlich Informationen über das Wetter, Wochentag, Monat und Feiertage. Der Monat wirkt sich nicht nur auf die Durchschnittliche Temperatur aus, sondern auch auf die Ladeanfragen der Nutzer, wie in Abbildung 2.11a ersichtlich ist. Die Abbildung 2.11b ist in Bezug auf die Ladeanfragen pro Monat nicht aussagekräftig, jedoch zeigt sie, dass die Ladeanfragen stark von Uhrzeiten abhängig sind. Ein kleiner Anstieg ist zu erkennen bei Ladeanfragen ab 5Uhr, der sich bis 12 Uhr fortsetzt, bevor die Anfragen abflachen und nachmittags wieder zunehmen.

Der Bereich für EV-Last- und Ladeprognose ist gut erforscht und liefert ein solides Fundament auf dem diese Arbeit aufbaut. Die Methoden und Modelle, die in diesen Forschungsarbeiten verwendet werden, sind relevant für die Vorhersage von Ladebedarfen und Ladeanfragen, insbesondere in Hinblick auf die Unsicherheitsquantifizierung und äußere Einflussfaktoren wie Wetter, Wochentag und Sonntag und Feiertag. Die Ergebnisse dieser Studien zeigen, dass die Berücksichtigung von Unsicherheiten und externen Faktoren die Genauigkeit der Vorhersagen erheblich verbessern können.



(a) Durchschnittliche Ladeanfragen je Tageszeit, aufgeschlüsselt nach Monat.



(b) Ladeanfragen im Jahresverlauf, aufgeschlüsselt nach Wochentag.

Abbildung 2.11.: Zeitliche Muster der Ladeanfragen im verwendeten Datensatz (Sana-mi u. a. 2025).

Unsicherheits- und Quantilsprognose

Die Studie Amara-Ouali u. a. 2025 ist auch im Bereich der Unsicherheitsprognose relevant, da sie die Unsicherheit des Ladeverhaltens von Elektrofahrzeugen quantifiziert. Durch die Verwendung von probabilistischen Modellen wie GAMLSS oder QGAM werden Prognoseintervalle anstelle von Punktwerten erstellt, was eine umfassendere Darstellung der Unsicherheit ermöglicht. Eine weitere Studie (Ali u. a. 2024) befasst sich mit der Quantilsregression und deren Anwendung in der Vorhersage, gekoppelt mit einem Multi-Quantile Temporal Convolutional Network (MQ-TCN). Das Modell erzielte einen 93,62% hohen Intervallabdeckungswahrscheinlichkeit Score (PICP), der bis zu 96,88% steigt, mit nur zwei Wochen Trainingsdaten. Die Studie zeigt, dass die Quantilsregression in Kombination mit einem gut trainierten und konfigurierten neuronalen Netzwerk eine effektive Methode zur Vorhersage von Unsicherheiten in Mobilitätsdaten darstellt. Die Ergebnisse dieser Studien sind besonders relevant für die vorliegende Arbeit, da sie die Bedeutung der Quantilsregression und der Unsicherheitsprognose in der individuellen Fahrzeugnutzungsprognose unterstreichen.

Mobilitäts- und Nutzungsforecast

Ein zentraler Befund dieser Sektion betrifft die Heterogenität individuellen Fahrverhaltens. Ji u. a. (2023) werten über vier Millionen Fahrten von 3743 privaten Fahrzeugführern in sechs US-Metropolregionen aus und zeigen, dass sich die Fahrer in fünf deutlich verschiedene Typen gliedern lassen (Tabelle 2.4), die sich vor allem in ihrer Pendelregelmäßigkeit und im Anteil nicht-arbeitsbezogener Fahrten unterscheiden. Diese Heterogenität erklärt, warum ein einzelnes, übergreifendes Modell dem Individualverkehr nur begrenzt gerecht wird und motiviert eine individuelle beziehungsweise quellenspezifische Modellierung.

Fahrertyp	Charakterisierung
Homebodies	Wenigfahrer, die nur kurz und selten das Haus verlassen; geringe Mobilität.
Gig Drivers	Fahrer mit hohem, auftragsgetriebenem Fahraufkommen (z. B. Liefer- und Fahrdienste) ohne festen Arbeitsort.
Movers	Individuen mit breit gestreutem Fahrverhalten und vielen wechselnden Zielen, ohne klares Muster.
Typical Drivers	Durchschnittsfahrer mit einer Mischung aus Arbeits- und Freizeitfahrten und mittlerer Regelmäßigkeit.
Work-focused Commuters	Berufspendler mit stark regelmäßigem Pendelmuster, wenig nicht-arbeitsbezogenen Fahrten und hoher Vorhersagbarkeit.

Tabelle 2.4.: Fünf Fahrertypen des Individualverkehrs nach Ji u. a. (2023).

3. Methodik

Das Ziel der Methodik ist es, die zuvor genannten Forschungsfragen in Abschnitt 1.4 zu beantworten. Um dies zu erreichen, werden in Abschnitt 3.2 und Abbildung 3.2 die methodischen Schritte beschrieben, die dafür durchgeführt werden.

Datenquellen Die Varietät und Qualität der Datenquellen ist ein entscheidender Faktor für die Leistungsfähigkeit von Machine-Learning-Modellen. In dieser Arbeit werden Datenquellen verwendet, die unterschiedliche Aspekte des Problems abdecken, wovon jede auf ihre Eignung hin untersucht und in ein einheitliches Schema transformiert wird, um ein durchgehendes Feature-Set zu gewährleisten. Die Datenquellen werden in Abschnitt 3.3 detailliert beschrieben, einschließlich ihrer Herkunft, Struktur und der Art der enthaltenen Informationen. Die Auswahl der Datenquellen (ersichtlich in Abbildung 3.2) basiert auf ihrer Relevanz für die Forschungsfragen und ihrer Fähigkeit, ein möglichst breites Spektrum an Mobilitätsfragen abzudecken.

Modellfamilien Auf Grund der Studienlage und der darin stark repräsentierten Modellfamilien werden in dieser Arbeit drei verschiedene Modelle verwendet, die jeweils unterschiedliche Eigenschaften aufweisen. Die Modelle sind Random Forest, LightGBM und LSTM – welche in Abschnitt 2.3.8, Abschnitt 2.3.9 und Abschnitt 2.3.11 aufbereitet sind.

3.1. Forschungsdesign

Die Arbeit folgt einem quantitativ-experimentellen Forschungsdesign, das auf der Analyse von Daten und der Anwendung von Machine-Learning-Methoden basiert. Das Ziel ist es, die Forschungsfragen durch die systematische Gegenüberstellung von unterschiedlichen Datenquellen und Modellen zu beantworten. Da der Kern der Thesis die Erforschung eines Verfahrens ist, um die Prognosegüte von Modellen zu ermitteln, umfasst die Methodik ein kontrolliertes Experiment, die Evaluierung der

Modelle und die Durchführung einer Ablationsstudie mit numerischen Metriken und Prognosequantilen, um die Leistungsfähigkeit der Modelle zu bewerten.

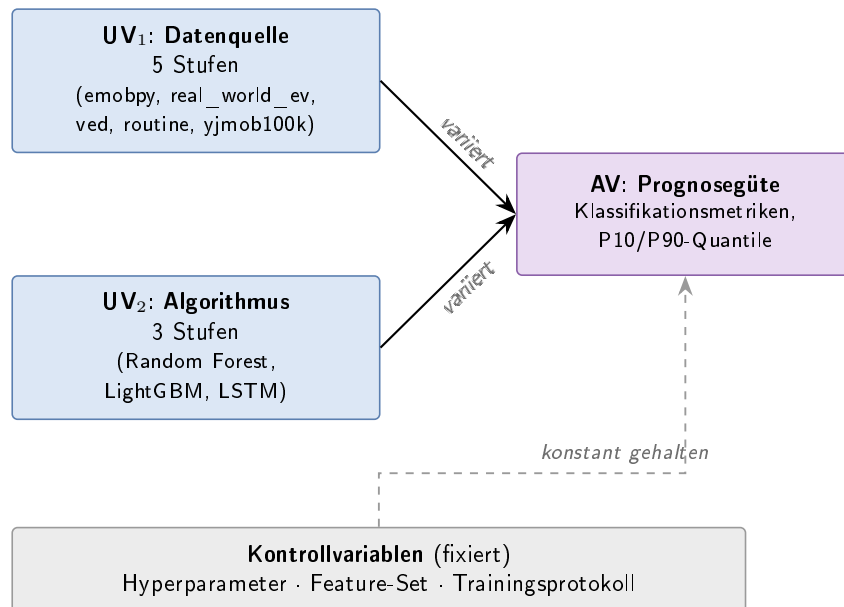


Abbildung 3.1.: Variablenstruktur des Forschungsdesigns: zwei veränderliche unabhängige Variablen (Daten, Algorithmus) und eine abhängige Variable (Prognosegüte). Konstante Hyperparameter und Trainingsprotokolle wirken als Kontrollvariablen.

Die unabhängigen Variablen in der Experimentreihe sind die verschiedenen Datensätze, die jeweils unterschiedliche Informationen enthalten und die unterschiedlichen Machine-Learning-Algorithmen. Die Hyperparameter werden je Modell einmalig festgelegt und während der Isolationsexperimente konstant gehalten; dadurch wirken sie, wie das Feature-Set und Trainingsprotokoll, als Kontrollvariablen (Abbildung 3.1). Die abhängige Variable ist die Prognosegüte, da diese von den unabhängigen Variablen beeinflusst wird. Für die Ablation werden die gelernten Modelle per Algorithmus- und Datenisolation untersucht, um die jeweiligen Effekte auf die Vorhersagegenauigkeit zu bestimmen. Die Ergebnisse werden statistisch analysiert, um die Qualität der Modelle auf Grund ihrer Datenbasis und ihres Algorithmus zu bewerten. Im Zuge dieses Aufbaus können, wie in Tabelle 3.1, folgende Hypothesen aufgestellt werden:

Tabelle 3.1.: Hypothesen zu den Forschungsfragen.

Nr.	Hypothese der Forschungsfrage
Mobilitätsprognose	
FF1	Durch die Kombination eines ausreichend komplexen Modells mit hochqualitativen Trainingsdaten kann das individuelle Mobilitätsverhalten zuverlässig vorhergesagt werden.
FF1.1	Der Vergleich verschiedener Verfahren des maschinellen Lernens zeigt unterschiedliche Vorhersagegenauigkeiten.
FF1.1.1	Die Datenqualität und -quelle haben einen erheblichen Einfluss auf die Prognosegüte, unabhängig vom gewählten Algorithmus.
FF1.2	Die wichtigsten Merkmale, die das individuelle Mobilitätsverhalten beeinflussen, sind u.a. der Tageszeitpunkt, die Wochentagsstruktur und die historischen Fahrten.
Mobilitätsdaten	
FF2	Anforderungen, die sich ergeben, sind u.a. Anspruch an Form, Qualität, Relevanz und Verwendbarkeit der Daten.
FF2.1	Aufbereitung wirkt sich im Optimalfall nicht auf die Nutzbarkeit der Daten aus; Fehlerhafte Daten können optional für Robustheit beibehalten werden.
FF2.2	Die Qualität der Daten lässt sich oberflächlich durch ihre Kohärenz, Vollständigkeit und Reichhaltigkeit bewerten. Tiefergehend lässt sich die Qualität der Daten anhand der Prognosequalität der Modelle interpretieren.

3.2. Untersuchungsablauf

Die Methodik umfasst die Auswahl und Implementierung von Datenaufbereitung und Machine-Learning-Modellen, die Gegenüberstellung der Modelle, sowie die Durchführung von Ablationsexperimenten, um die besten Modelle zu identifizieren. Die Methodik ist in Abbildung 3.2 schematisch dargestellt. Der Arbeitsfluss beginnt mit der Auswahl und Aufbereitung der Daten – in diesem Fall fünf verschiedene Datenquellen – welche in ein einheitliches Schema transformiert werden. Als nächstes werden die Daten auf ihre Qualität hin bewertet und relevante Features extrahiert.

Anschließend werden die Daten in Modelle eingespeist, die unterschiedliche Aufgaben erledigen: Klassifikator und Zeitreihenprognose. Die Modelle werden dann in einem experimentellen Design evaluiert, das sowohl die Isolation von Daten- und Algorithmen-Effekten als auch eine Ablationsstudie umfasst. Das Isolieren der Daten- und Algorithmen-Effekte ist ein wichtiger Aspekt der Methode, um sicherstellen zu können, dass die Ergebnisse auf die jeweiligen Einflussfaktoren zurückgeführt werden können. Abschließend werden die Ergebnisse bewertet und die Forschungsfragen beantwortet.

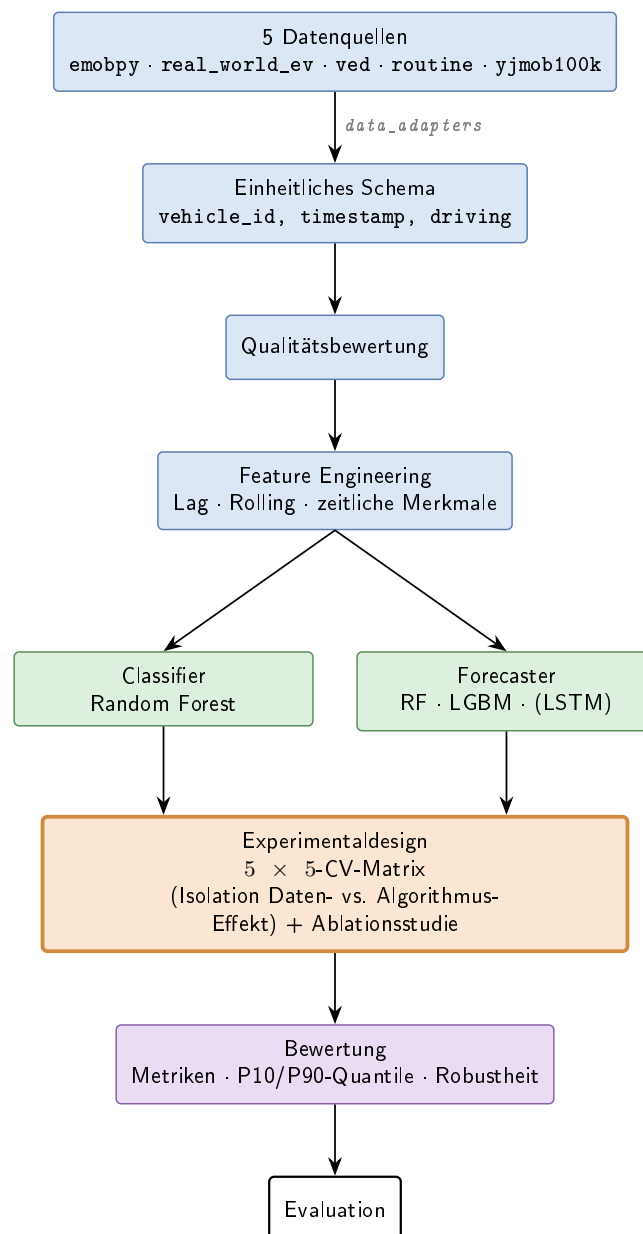


Abbildung 3.2.: Schematischer Workflow der Methodik: von den fünf Datenquellen über den geteilten Feature-Kern und die beiden Aufgabenmodelle bis zum Experimentaldesign und der Bewertung.

Die Arbeit lässt sich in drei große Teile gliedern:

- **Datenaufbereitung:** Das Erkundschaffen der Datenlandschaft, die Transformation der Daten in ein geeignetes Format und die Auswahl relevanter Features.
- **Machine Learning:** Die Auswahl und Implementierung verschiedener Machine-Learning-Modelle auf Grund ihrer unterschiedlichen Eigenschaften (leaf- und level-wise Wachstum, Bagging und Boosting, modellierte zeitliche Abhängigkeiten).
- **Ablationsexperimente:** Vergleich verschiedener Modelle hinsichtlich ihrer Wertigkeit (in diesem Fall: Vorhersagegenauigkeit, Robustheit, Zuverlässigkeit).

3.3. Datengrundlage und Quellen

Da die Daten einen großen Einfluss auf die Leistungsfähigkeit der Modelle haben, werden in dieser Arbeit fünf verschiedene Datenquellen verwendet, die unterschiedliche Aspekte des Problems abdecken. Jede Datenquelle wird auf ihre Eignung hin untersucht und in ein einheitliches Schema transformiert, sodass ein durchgehendes Feature-Set gewährleistet ist. Die Datenquellen werden hinsichtlich ihrer Herkunft, Struktur, Art der enthaltenen Informationen und Zweck ihres Gebrauchs beschrieben. Die Auswahl ist auf ihre Relevanz für die Forschungsfragen und ihr Potenzial, vorliegende Mobilitätsfragen zu beantworten, begründet. Die Datenquellen sind in Tabelle 3.2 zusammengefasst.

Tabelle 3.2.: Übersicht der fünf Datenquellen und ihrer identifizierenden Eigenschaften.

Quelle	Art	Träger	Umfang
emobpy	synthetisch	Fahrzeug	200 Fahrzeuge, 1 Jahr
real_world_ev	real	Fahrzeug	1 Fahrzeug, 1 Jahr
ved	real	Fahrzeug	383 Fahrzeuge, 1 Jahr
routine	synthetisch	Person	1 Person, dynamisch
yjmob100k	real	Person	25.000 Personen, 75 Tage

emobpy ist ein Open-Source-Python-Tool, das vom DIW Berlin (Deutsches Institut für Wirtschaftsforschung) entwickelt wurde, um synthetische Mobilitätsdaten zu

generieren (Gaete-Morales u. a. 2021). Die Daten werden auf Basis von realen Mobilitätsstatistiken und physikalischen Eigenschaften von Elektrofahrzeugen erzeugt. Für die Illustration des Tools wurde ein Profil von 200 Fahrzeugen über ein Jahr in Deutschland simuliert. **emobpy** liefert ein **in_use**-Flag, weswegen die Identifikation des Fahrverhaltens direkt aus den Daten abgeleitet werden kann. Die Aktivitätsrate liegt im Median bei 9,66 %.

real_world_ev ist ein Datensatz, der über ein Jahr hinweg Fahrdaten eines einzelnen Elektrofahrzeugs sammelt, um dessen Batterieverhalten besser verstehen zu können (Pozzato u. a. 2023). Die Daten liegen im **.mat**-Format vor, weswegen diese in Python eingelesen, in ein einheitliches Schema transformiert und für die Modellierung in eine **.csv**-Datei exportiert werden. Da es keine Flag wie **in_use** gibt, die anzeigt, ob das Fahrzeug tatsächlich in Benutzung ist, wird die Aktivität über eine Stromschwelle von 5 A (Fahren) sowie den Ladeprozess bestimmt; **driving** umfasst bei dieser Quelle also Fahren *und* Laden. Da der Datensatz nur ein Fahrzeug umfasst, liegt dessen Aktivitätsrate bei 5,54 %.

ved (Vehicle Energy Dataset) ist ein Datensatz, der durch die Universität Michigan, das Argonne National Lab und das Idaho National Lab bereitgestellt wird (Oh u. a. 2022). Die Daten stammen von 383 Fahrzeugen, welche in Ann Arbor, Michigan, USA über ein Jahr hinweg Fahrdaten gesammelt haben. Die OBD-II-Signale werden hochfrequent *während der Fahrt* protokolliert; für Parkzeiten liegen keine Messpunkte vor, sodass sich die Fahraktivität unmittelbar aus der Existenz eines Eintrags ableiten lässt und ein zusätzlicher Aktivitätsfilter entfällt. Der Datensatz hebt sich mit seiner stark abweichenden Aktivitätsrate ab; der Median liegt bei 0,99 %, die Spanne jedoch bei 0–100 %. Diese ergibt sich daraus, dass manche Fahrzeuge keine aktiven Stunden aufzeichnen, andere nur einen einzelnen Tag, der ausschließlich aus Fahrten besteht.

routine ist ein stark regelbasierter Algorithmus, um synthetische Daten zu generieren. **routine** wurde im Verlauf dieser Arbeit entwickelt, um eine an deutsche Arbeitsalltage angelehnte Alternative zur Verfügung zu haben. Die Tagesabläufe richten sich stark nach dem üblichen Muster:

- **Arbeit:** 3-Schicht-Modell
- **Außerordentliches:** Trips, die nicht zur Arbeit gehören. Freizeit, Einkauf, andere Aktivitäten
- **Wochenende:** Keine oder weniger Arbeit. Unregelmäßiges Fahrverhalten.

- **Sonn- und Feiertage:** Keine Arbeit, wenige Freizeitmöglichkeiten. Somit geringstes Verkehrsaufkommen

Beim **routine**-Algorithmus ist die Aktivitätsrate variabel; beim zuletzt generierten Datensatz liegt sie bei circa 25 %. Der Algorithmus erzeugt einzelne Personen, die einen nachvollziehbaren Tagesablauf verfolgen.

yjmob100k hebt sich von den anderen Datensätzen ab, da er Bewegungsdaten von Smartphones statt von Fahrzeugen selbst erfasst (Yabe u. a. 2024). Bildlich gesprochen wird ein Raster mit 500 m Kantenlänge, also 250.000 m² Fläche, auf die Karte gelegt; eine Bewegung wird nur dann erkannt, wenn eine Grenze zu einer anderen Rasterzelle überschritten wird. Da als Bewegung jeder Zellenwechsel gilt – inklusive nicht-motorisierter Wege, die in einer Großstadt (hier in Japan) häufig zu Fuß zurückgelegt werden – ergibt sich eine höhere Median-Aktivitätsrate von 35,33 % bei einer Spanne von 3,9–84,72 %. Diese Rate ist daher nicht direkt mit den fahrzeugbasierten Quellen vergleichbar.

Die Aufbereitung der Daten erfolgt in einem einheitlichen Muster – durch einen Adapter. Die enthaltenen Informationen werden auf das Wesentliche gekürzt: Fahrzeugidentifikationskennung und die Information, ob sich das Fahrzeug im Moment im geparkten oder fahrenden Zustand befindet. Datensätze, die keine konkrete Information darüber enthalten, ob ein Fahrzeug sich fortbewegt, werden durch andere Methoden identifiziert – bei **real_world_ev** wird beispielsweise die momentane Stromstärke verwendet, bei **ved** genügt die Existenz eines Eintrags, da nur Fahrten dokumentiert werden.

Die Qualität der Datensätze lässt sich durch die Prognosegüte evaluieren, denn selbst wenn der Datensatz keine Fehler aufweist und laut Überprüfung optimal wäre, bestünde die Möglichkeit, dass die Varianz über die Fahrzeuge hinweg zu groß ist und somit für Anforderungen der individuellen Mobilitätsprognose nicht ausreicht.

Da die individuelle Mobilitätsprognose für möglichst viele unterschiedliche Personen zuverlässig funktionieren soll, ist es von Vorteil, grundverschiedene Quellen zum Trainieren und Testen zu verwenden: Nur so lässt sich prüfen, ob das Verfahren über heterogene Datenbedingungen hinweg generalisiert. Die Quellen unterscheiden sich in der Art der Erhebung beziehungsweise Erzeugung, dem Ort der Studie, der Anzahl der Subjekte, den Rahmenbedingungen und dem Erhebungszeitraum. Gerade wegen dieser starken Heterogenität (stark unterschiedliche Definition von Aktivität) sind die absoluten Aktivitätsraten der Quellen nur eingeschränkt direkt vergleichbar. Die Überführung in ein einheitliches Schema und das isolierende Experimentaldesign

stellen sicher, dass Daten- und Algorithmuseffekt dennoch getrennt beurteilt werden können.

3.4. Feature Engineering

Das stündliche Driving-Raster ist die Eingangs- und Zielstruktur, aus deren Vergangenheit Merkmale abgeleitet werden. Das Driving-Raster sorgt dafür, dass die Daten in einem festen Schema aufbereitet werden, um damit in einem geregelten Muster arbeiten zu können.

Aus diesem Raster werden die folgenden Features abgeleitet, die als Eingangsgrößen für die Modelle dienen:

- Hour, Weekday, Is_weekend
- Lag_1, Lag_2, Lag_24, Lag_168
- Rolling_mean_24, Rolling_mean_168

Die aufgelisteten Features sind die Basis für die Modelle, um die Vorhersage zu treffen. Die Features Hour, Weekday und Is_weekend sind zeitliche Markierungen, um die Tageszeit, den Wochentag und das Wochenende zu identifizieren. Die Identifikation dieser Merkmale ist wichtig, da das Mobilitätsverhalten stark von diesen Faktoren abhängt. So ist ein Fahrverhalten beispielsweise Montag morgens zur Arbeit, jedoch an einem Sonntag nicht. Auf Sonn- und Feiertage wird nicht genauer als eigenes Merkmal eingegangen, da diese in den Quellen nicht einheitlich erfasst werden und somit nicht zuverlässig verwendet werden können.

Die Lag-Features (Lag_1, Lag_2, Lag_24, Lag_168) geben den Aktivitätszustand zu einem jeweils früheren Zeitpunkt wieder. Die Wahl dieser vier Verzögerungen folgt den stärksten Faktoren der Periodizität: Lag_1 und Lag_2 erfassen die kurzfristige Autokorrelation, da sich einzelne Fahrten typischerweise über mehrere aufeinanderfolgende Stunden erstrecken. Lag_24 entspricht derselben Stunde des Vortages und bildet damit die Tagesperiodik ab, etwa wiederkehrende Pendelzeiten. Lag_168 entspricht derselben Stunde der Vorwoche und erfasst die Wochenperiodik, insbesondere den Unterschied zwischen Werktagen und Wochenende. Diese Merkmale sind notwendig, weil baumbasierte Modelle keine zeitliche Ordnung ihrer Eingaben kennen. Das bedeutet, dass jede Zeile unabhängig betrachtet wird, sodass zeitliche Bezüge nicht selbstständig gebildet werden können und explizit als Merkmale bereitgestellt werden müssen.

Rolling-Mean-Features (Rolling_mean_24, Rolling_mean_168) repräsentieren den Durchschnitt der Aktivität über die letzten 24 beziehungsweise 168 Stunden. Diese Features helfen dabei, das Aktivitätsmuster im Mobilitätsverhalten zu erkennen, die für die Vorhersage der zukünftigen Aktivität relevant sind.

Bei der Nutzung von historischen Zeitreihendaten ist es wichtig, deren chronologische Abfolge zu berücksichtigen. Ein potentiell Problem, das bei der Nutzung von solchen Daten auftreten kann, ist das sogenannte *Data Leakage*. Data Leakage tritt auf, wenn Informationen aus der Zukunft in das Modelltraining einfließen, was zu einer unrealistischen Leistungsbewertung führt. Um dieses Problem zu vermeiden, werden die Daten in Trainings- und Testsets aufgeteilt, wobei die chronologische Reihenfolge strikt eingehalten wird. Das bedeutet, dass das Modell nur auf Basis der historischen Daten trainiert wird und keine Informationen aus der Zukunft erhält. Durch das Unterbinden von willkürlichen Daten-Splits wird sichergestellt, dass die Leistungsbewertung des Modells realistisch ist und die Vorhersagegüte nicht durch Data Leakage verfälscht wird.

3.5. Modellauswahl

Bei der Modellauswahl ist vor Allem die Heterogenität der Modelle entscheidend, um die Forschungsfragen zu beantworten. Die Modelle unterscheiden sich in ihrer Funktionsweise, ihren Stärken und Schwächen, sowie in ihrer Fähigkeit mit unterschiedlichen Arten von Daten umzugehen. Die Auswahl der Modelle basiert auf der Literaturrecherche und den spezifischen Anforderungen der Forschungsfragen.

Random Forest Modelle zeichnen sich durch ihre Fähigkeit aus, komplexe nichtlineare Zusammenhänge zu modellieren und dabei eine hohe Robustheit gegenüber Rauschen in den Daten aufzuweisen. Durch das Bagging-Verfahren, bei dem mehrere Entscheidungsbäume auf unterschiedlichen Stichproben der selben Trainingsdaten trainiert werden, können Random Forests eine große Menge an Entscheidungsbäumen kombinieren, die jeweils unterschiedliche Aspekte der Daten erfassen. Die Vorhersagen der einzelnen Bäume werden dann aggregiert, um eine finale Vorhersage zu treffen.

LightGBM ist ein Gradient-Boosting-Framework, das auf Entscheidungsbäumen basiert und für seine hohe Effizienz und Skalierbarkeit bekannt ist. Es verwendet eine spezielle Technik namens *Leaf-wise Growth*, die es ermöglicht, tiefere Bäume zu erzeugen und dadurch komplexere Muster in den Daten zu erfassen. LightGBM ist

besonders gut geeignet für große Datensätze und kann sowohl Klassifikations- als auch Regressionsaufgaben durchführen.

LSTM Beide vorgenannten Verfahren sehen die Zeitreihe ausschließlich in Gestalt der vorab konstruierten Merkmale; die zeitliche Ordnung geht dabei verloren. Ein rekurrentes Netz (Abschnitt 2.3.11) verarbeitet die Historie dagegen als Sequenz und kann Abhängigkeiten selbst ableiten. Um beide Effekte – Architektur und Repräsentation – zu trennen, werden zwei Varianten geführt: Die erste erhält exakt denselben Merkmalssatz wie Random Forest und LightGBM und misst gegen diese den reinen *Architektureffekt*. Die zweite erhält ein Fenster der letzten 168 rohen Stundenwerte und misst gegen die erste den *Repräsentationseffekt*. Die Netze sind bewusst klein dimensioniert, da die Aktiv-Rate je nach Quelle nur 5–10 % beträgt und größere Netze unmittelbar überanpassen.

Für die rekurrenten Varianten wird der Umfang einer Quelle vorab in *Fahrzeugwochen* bemessen, nicht in Zeilen: Da sich die gleitenden Fenster fast alle Zeitschritte teilen, wächst die Zahl der Beobachtungen mit der Fensterzahl, die Zahl der tatsächlich unabhängigen Verlaufsmuster jedoch nur mit der abgedeckten Zeitspanne. Quellen unterhalb von 100 Fahrzeugwochen werden daher nicht als Trainingsquelle des LSTM zugelassen und bleiben in der Isolationsmatrix als Trainingszeile leer; als *Testquelle* bleiben sie uneingeschränkt erhalten. Die Schranke ist vor Sichtung der Ergebnisse festgelegt worden, um eine nachträgliche Auswahl zugunsten günstiger Werte auszuschließen.

3.6. Experimentaldesign

Modelle werden in einem Experimentaldesign auf Basis der verschiedenen Datenquellen bei einem 80% Training und 20% Test trainiert und getestet, um die Auswirkungen der Datenqualität und -quelle auf die Prognosegüte zu untersuchen. Im ganzen gibt es fünf verschiedene Datenquellen, die jeweils unterschiedliche Informationen enthalten. Für jede Datenquelle werden die Modelle trainiert und anschließend mit jeder Datenquelle getestet. Durch dieses Verfahren lässt sich die Prognosegüte der Modelle anhand von jeder Datenquelle darstellen – jedes Modell schneidet durch seine verschiedenen Trainingsdaten unterschiedlich ab (Abbildung 5.1 und Abbildung A.1). Da sich sowohl die Architekturen der Modelle als auch Datenquellen unterscheiden, bietet sich die Möglichkeit sowohl die Güte der Daten als auch die Güte der Modelle gegenüberzustellen, was sich durch eine Kreuzvalidierung darstellen lässt.

Zusätzlich zur Kreuzvalidierung wird eine Ablationsstudie durchgeführt, um die Auswirkungen der Datenmerkmale auf die Prognosegüte zu untersuchen. Hierbei werden bestimmte Merkmale der Datenquelle ignoriert oder gezielt isoliert, um den Einfluss der einzelnen Merkmale zu untersuchen. Merkmale wie die Tageszeit (`lag_X`), der Wochentag, welcher durch den Timestamp abgeleitet wird und die historischen Fahrten werden dabei gezielt untersucht; der Einfluss wird als Abfall der PR-AUC gegenüber dem vollen Merkmalssatz gemessen. Dadurch lässt sich genauer bestimmen, welche Merkmale der Datenquelle einen signifikanten Einfluss auf die Prognosegüte haben und welche nicht.

Die Versuche kennzeichnen sich durch drei Merkmale:

- **Datenquelle:** Die Daten werden einmalig geteilt und für alle Modelle identisch verwendet, um Vergleichbarkeiten und Differenzierungen zu gewährleisten.
- **Architektur:** Drei Verfahren (Random Forest, LightGBM, LSTM) werden auf Basis der gleichen Daten trainiert und getestet, um den Algorithmuseffekt zu isolieren.
- **Seeds:** Bedeutet eine unterschiedliche und zufällige Initialisierungen (bei baumbasierten Modellen Bootstrap und Merkmalsauswahl, beim LSTM Gewichte und Mini-Batch-Reihenfolge), sodass jede Zelle der Kreuzvalidierung Mittelwert *und* Standardabweichung aus fünf Durchläufen trägt; erst deren Streuung entscheidet, ob ein Unterschied berichtbar ist.

Es werden fünf Datenquellen verwendet, um fünf Trainings- und Testsets zu bilden, die jeweils unterschiedliche Informationen enthalten. Diese Daten bleiben für alle Architekturen und Modelle identisch, sodass die Ergebnisse der Modelle direkt miteinander vergleichbar sind. Bei der Untersuchung der Datenquellen wird die Qualität anhand der Architekturen und Modelle bewertet. Nun werden aus den drei Architekturen vier verschiedene Modelle (RF, LGBM, LSTM: Sequentiell, Tabellarisch) gebildet, die jeweils unterschiedliche Eigenschaften aufweisen (siehe Grundlagen Abschnitt 2.3.8, Abschnitt 2.3.9, Abschnitt 2.3.11) – Bei den LSTM Modellen wird jedoch die Differenz zwischen dem sequentiellen und tabellarischen Modell als Repräsentation verwendet. Da die sequentiellen Varianten mindestens 100 Fahrzeugwochen voraussetzen (Abschnitt 3.5), werden sie nur auf `emobpy` und `yjmob100k` trainiert; die übrigen Quellen bleiben als Trainingszeile leer, als Testquelle jedoch erhalten. Je Seed ergeben sich so 14 Modelle, jedes Modell wird gegen alle fünf Testquellen getestet, sodass 70 Auswertungen pro Seed entstehen. Da fünf Seeds berechnet werden, ergeben sich insgesamt 350 Auswertungen. Die Ergebnisse werden dann gemittelt und in Form einer Kreuzvalidierungsmatrix dargestellt, die die Prognosegüte der Modelle auf Basis der verschiedenen Datenquellen zeigt. Die

14 Modelle ergeben sich durch: fünf Modelle für Random Forest, fünf Modelle für LightGBM und vier Modelle für LSTM (zwei tabellarisch und zwei sequentiell).

3.7. Bewertung

Bei der Bewertung werden verschiedene Metriken verwendet, um die Vorhersagequalität der Modelle zu bewerten. Dadurch, dass Mobilitätsdaten stark unbalanciert sind, ist die Wahl der Metriken entscheidend, um die Prognosegüte korrekt zu bewerten.

Accuracy ist eine Metrik, die den Anteil der korrekt vorhergesagten Werte an der Gesamtzahl der Vorhersagen misst. Hierbei stehen die Kürzel für: TP (True Positives), TN (True Negatives), FP (False Positives) und FN (False Negatives).

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (3.1)$$

In diesem Fall ist die Accuracy nicht die beste Metrik, da die Daten stark unbalanciert sind und die Accuracy daher nicht die tatsächliche Vorhersagequalität widerspiegelt. Accuracy offenbart sich als unzureichend, da sie auf Grund der niedrigen Aktivitätsrate zwar oft eine hohe Genauigkeit aufweist, jedoch die Vorhersagequalität der positiven Klasse nicht korrekt widerspiegelt.

Precision wird dazu verwendet, um die Genauigkeit der positiven Vorhersagen zu messen. Sie gibt an, wie viele der als positiv vorhergesagten Werte tatsächlich positiv sind.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (3.2)$$

Recall dient dazu, die Fähigkeit des Modells zu messen, alle relevanten positiven Werte zu identifizieren. Der Unterschied zur Precision besteht darin, dass der Recall den Anteil der tatsächlich positiven Werte misst, die korrekt als positiv vorhergesagt wurden.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (3.3)$$

F_β -Score fasst Precision und Recall zu einem gewichteten harmonischen Mittel zusammen. Der Faktor β legt fest, um welchen Faktor der Recall gegenüber der Precision stärker gewichtet wird: $\beta > 1$ betont den Recall (verpasste Fahrten wiegen schwerer), $\beta < 1$ die Precision (Fehlalarme wiegen schwerer).

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (3.4)$$

In dieser Arbeit ist $\beta = 1$ gesetzt, sodass Fehlalarme und verpasste Fahrten gleich gewichtet werden. Aus der Gewichtung ergibt sich als Spezialfall der gebräuchliche F1-Score (Rijsbergen 1979). Die Metrik ist relevant, um zu überblicken, welche Modelle relativ besser sind.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.5)$$

PR-AUC (Precision-Recall Area Under Curve) kombiniert Precision und Recall in einer Kurve, welche die Leistung des Modells über verschiedene Schwellenwerte hinweg darstellt. Der Flächeninhalt unter der Kurve (AUC) gibt einen aggregierten Wert für die Modellleistung an. Ein höherer AUC-Wert deutet auf eine bessere Modellleistung hin. Mit dem PR-AUC wird die absolute Vorhersagequalität bewertet und, ob die Imbalance fair berücksichtigt wird. Berechnet wird die Fläche als Average Precision, also als schwellenweise gewichtete Summe der Precision-Werte P_n , wobei die Gewichte der Recall-Zuwachs $R_n - R_{n-1}$ zwischen aufeinanderfolgenden Schwellen sind. Die Zufalls-Baseline entspricht der Aktiv-Rate der positiven Klasse.

$$\text{PR-AUC} = \sum_n (R_n - R_{n-1}) \cdot P_n \quad (3.6)$$

ROC-AUC misst als Rangmaß die Wahrscheinlichkeit, dass einer zufälligen Fahr-Stunde eine höhere Wahrscheinlichkeit zugewiesen wird als einer zufälligen Park-Stunde; sie wird nur ergänzend geführt, da sie bei starker Imbalance optimistisch ausfällt.

Brier-Score misst die Genauigkeit der probabilistischen Vorhersagen eines Modells, indem er den mittleren quadratischen Abstand zwischen der vorhergesagten Wahrscheinlichkeit p_i und dem tatsächlichen Ausgang $y_i \in \{0, 1\}$ über alle N Beobachtungen bildet. Ein niedrigerer Wert bedeutet eine niedrigere Abweichung und ob dem System vertraut werden kann.

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2 \quad (3.7)$$

4. Implementierung

Dieses Kapitel beschreibt und begründet die Implementierung des Systems. Es werden funktionale und nicht-funktionale Anforderungen, die Systemarchitektur, die Technologieauswahl, die Umsetzung der Kernkomponenten sowie die Herausforderungen bei der Implementierung erläutert. Das Ziel ist es, einen umfassenden Überblick zu dem Entwicklungsprozess und den getroffenen Entscheidungen zu geben, um die Nachvollziehbarkeit und Reproduzierbarkeit der Arbeit zu gewährleisten.

4.1. Anforderungen

In Tabelle 4.1 und Tabelle 4.2 werden die funktionalen und nicht-funktionalen Anforderungen an das System beschrieben. Diese Anforderungen bilden die Grundlage für die Implementierung und dienen als Referenzpunkt für die Bewertung des Systems. Die Anforderungen werden anhand von Titel, Kürzel, Kurzbeschreibung, Typ (funktional/nicht-funktional) und Priorität dargestellt. Die Priorität wird in drei Stufen unterteilt: *hoch*, *mittel* und *niedrig*. Die Funktionalen Anforderungen beschreiben die spezifischen Funktionen und Fähigkeiten, die das System erfüllen muss, während die nicht-funktionalen Anforderungen die Qualitätsmerkmale und Leistungsaspekte des Systems betreffen.

Tabelle 4.1.: Anforderungen an Daten und System

Titel	Kürzel	Kurzbeschreibung	Typ	Priorität
Daten				
Recherche	R-1.0	Daten für die Erstellung von Modellen werden erfasst, auf das richtige Format überprüft und bereitgestellt.	F	hoch
Aufbereitung	R-1.1	Daten werden aufbereitet und in das richtige Format gebracht: Informationsgehalt und -qualität, Granularität. Daten müssen in Schema <code>vehicle_id</code> , <code>timestamp</code> , <code>driving</code> passen.	F	hoch
Datenvielfalt	R-1.2	Mehr als zwei aussagekräftige Datensätze, für einen umfangreichen Vergleich.	NF	hoch
Speicherung	R-1.3	Daten werden dauerhaft persistiert.	F	niedrig
Leakage-Resistent	R-1.4	Die Daten müssen, durch einen passenden Algorithmus, gegen Leakage geschützt sein.	NF	hoch
Klassifikationslabel	R-1.5	Die Daten müssen ein Klassifikationslabel enthalten, das die Aktivität des Fahrzeugs beschreibt.	NF	hoch
Individuelle Fahrzeug-IDs	R-1.6	Die Daten müssen individuelle Fahrzeug-IDs enthalten, um die Aktivität jedes Fahrzeugs individuell beobachten zu können.	NF	hoch
System				
Verfügbarkeit	R-2.0	Gewährleistung der Funktionalitäten.	NF	hoch
Replikation	R-2.1	Versuche sollten deterministisch sein und replizierbar.	NF	mittel
User Interface	R-2.2	Eine Benutzeroberfläche die zur vereinfachten Bedienung und Darstellung des Systems dient.	F	niedrig
Logging	R-2.3	Bearbeitungsschritte, Resultate und Fehler werden protokolliert.	NF	mittel

Tabelle 4.2.: Anforderungen an die Machine-Learning-Modelle

Titel	Kürzel	Kurzbeschreibung	Typ	Priorität
Machine Learning Modelle				
Modellauswahl	R-3.0	Modelle die für das Aufgabengebiet geeignet sind.	F	hoch
Modellvergleich	R-3.1	Ein umfassender Vergleich der ausgewählten Modelle.	F	hoch
Prognosegüte	R-3.2	Die Fähigkeit der Modelle, zukünftige Werte präzise vorherzusagen.	NF	hoch
Modellevaluierung	R-3.3	Die Bewertung der Leistung der Modelle anhand einer 5x5 Kreuzvalidierung	F	hoch
Imbalance	R-3.4	Die Modelle müssen mit stark unausgeglichenen Klassen umgehen können.	NF	hoch

4.2. Systemarchitektur

Die Systemarchitektur ist in Abbildung 4.1 dargestellt. Sie zeigt die statische Struktur des Systems, einschließlich der Hauptkomponenten und deren Beziehungen. Die Systemarchitektur ist in unterschiedliche Schichten unterteilt, die jeweils spezifische Aufgaben erfüllen. Der Datenadapter, der verantwortlich ist für das Laden und Vorverarbeiten der Daten aus verschiedenen Quellen – da diese unterschiedliche Formate und Strukturen aufweisen. Diese werden dann an die Feature-Engineering-Komponente weitergeleitet, die relevante Merkmale extrahiert und für die Modellierung vorbereitet. Die Klassifikator-Komponente ist für die Vorhersage der Aktivität des Fahrzeugs zuständig, während der Forecaster die zukünftige Aktivität anhand von Monte-Carlo Verteilungen prognostiziert. Die Informationen werden an das Backend weitergeleitet, das die Persistenz der Resultate und Modelle – auf der Persistenzebene – sicherstellt. Das Backend stellt auch Schnittstellen für die Kommunikation mit dem Frontend bereit, sodass die Ergebnisse visualisiert und analysiert werden können. Dadurch, dass die Systemarchitektur modular aufgebaut ist, lassen sich einzelne Komponenten unabhängig voneinander entwickeln.

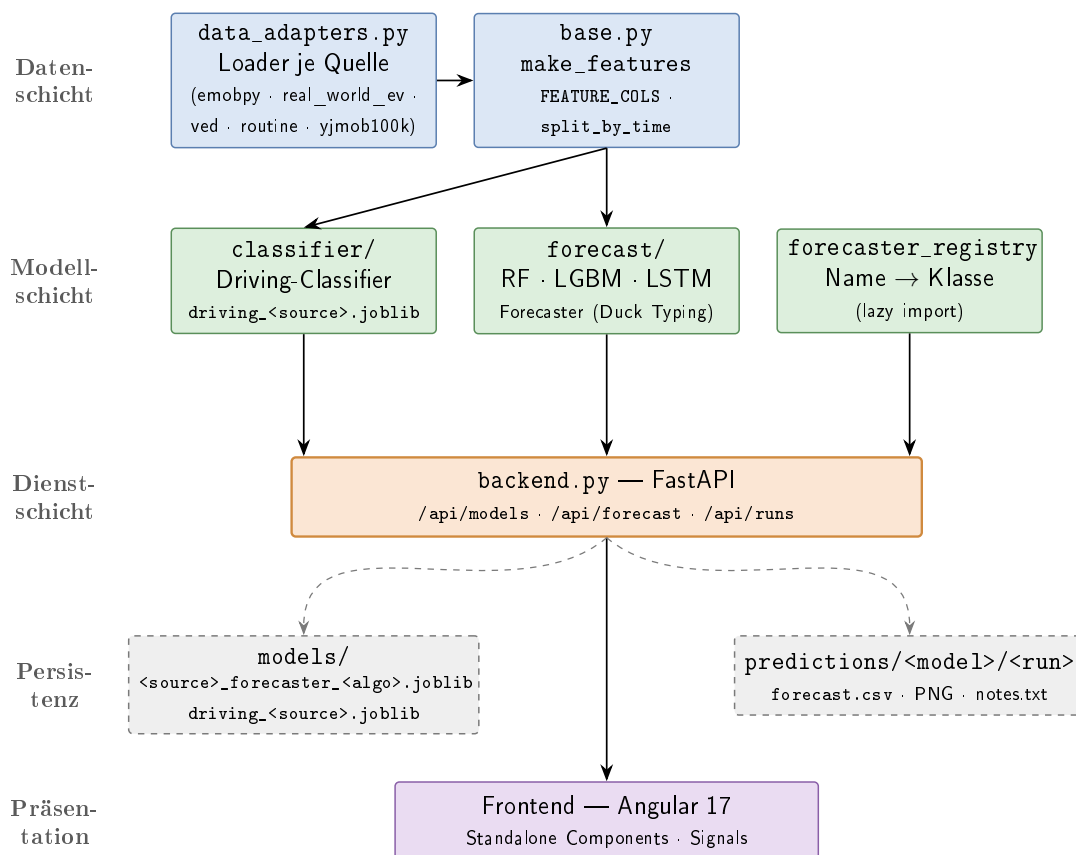


Abbildung 4.1.: Statische Komponentenstruktur: Datenadapter, Feature-Engineering, Klassifikator, Forecaster, Persistenz und Schnittstellen.

Der Arbeitsfluss des Systems lässt sich in Abbildung 4.2 nachvollziehen. Bei den Komponenten lassen sich fünf Komponenten identifizieren, die miteinander interagieren; die Benutzeroberfläche (Frontend) – oder die Kommandozeile – konsultiert mit einer Anfrage das Backend, das mit den erhaltenen Informationen in der Persistenzebene die Modelle und Daten abrufen. Mit den abgerufenen Informationen stößt das Backend die Vorhersage beim Forecaster an, der die Prognose vornimmt und die Ergebnisse als Datengerüst zurückgibt. Bei dem Datengerüst handelt es sich um eine strukturierte Sammlung von Vorhersagen, die die zukünftige Aktivität des Fahrzeugs beschreibt, darunter die Wahrscheinlichkeit, dass das Fahrzeug aktiv ist, sowie die prognostizierten Zeiträume der Aktivität. Das Backend leitet die Ergebnisse sowohl an die Persistenzebene zur Speicherung als auch an das Frontend zur Visualisierung weiter. Die Visualisierung der Ergebnisse erfolgt in Form von Diagrammen, Tabellen und anderen Darstellungen, die es dem Benutzer ermöglichen, die Vorhersagen zu interpretieren und fundierte Entscheidungen zu treffen.

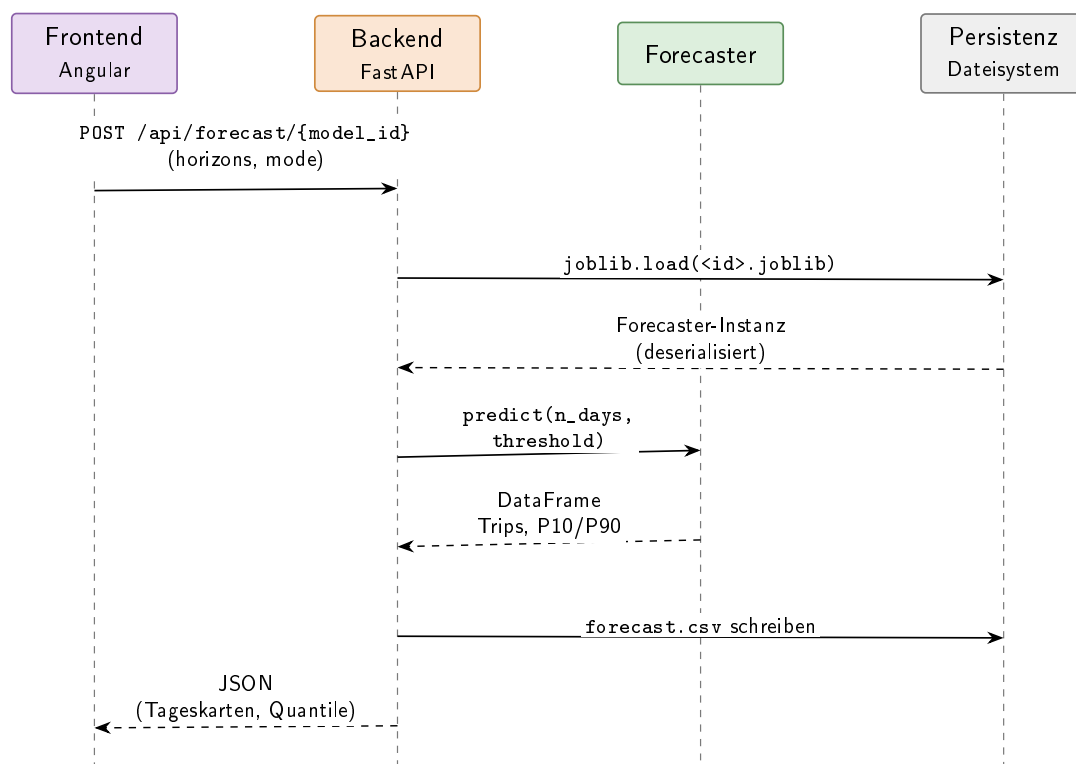


Abbildung 4.2.: Dynamischer Ablauf eines Forecast-Requests vom Frontend über das Backend und Persistenz bis zum Forecaster und dessen Aufbereitung und Darstellung der Ergebnisse.

Bei der Persistierung der Modelle und Resultate der Prognosen wird ein hierarchisches Verzeichnislayout verwendet, das in Abbildung 4.3 dargestellt ist. Die Modelle werden in einem separaten Verzeichnis gespeichert, während die Vorhersagen in Unterverzeichnissen nach Modell, Run und Zeitstempel organisiert sind. Dieses Layout ermöglicht

eine einfache Navigation und Verwaltung der Daten, da die Modelle und Vorhersagen klar voneinander getrennt sind. Die hierarchische Struktur der Daten erleichtert nicht nur die Identifizierung und den Zugriff auf die relevanten Informationen, sondern unterstützt auch Eindeutigkeit durch Run-IDs und Zeitstempel. Dadurch kann in der Benutzeroberfläche eine nachvollziehbare Darstellung der bisherigen Prognosen und Modelle gewährleistet werden, was die Analyse und Interpretation der Ergebnisse erleichtert.

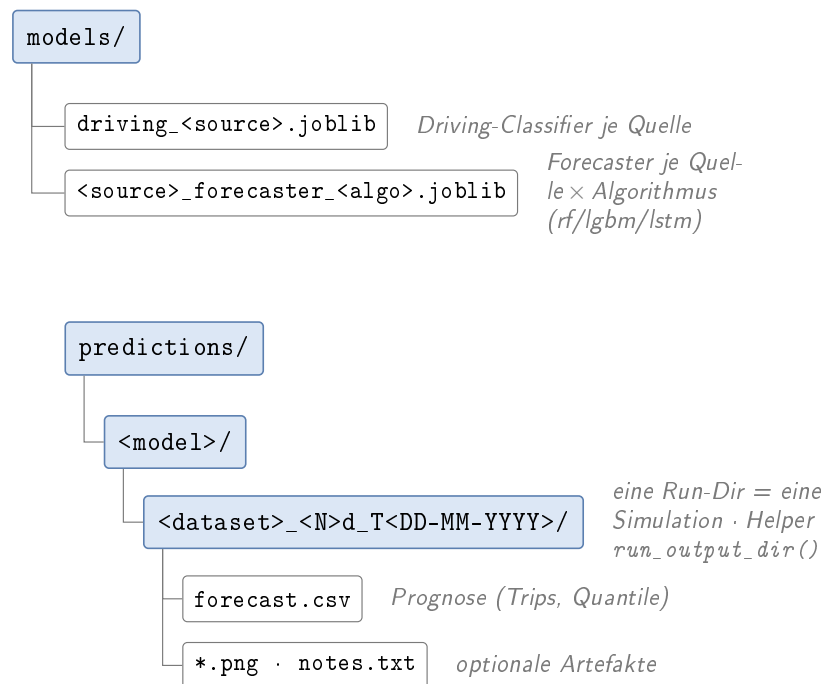


Abbildung 4.3.: Persistenzlayout des Systems: Modelle und Vorhersagen werden in einer hierarchischen Verzeichnisstruktur gespeichert, die eine einfache Navigation und Verwaltung der Daten ermöglicht. Modelle liegen gesondert von den Vorhersagen, die in Unterverzeichnissen nach Modell, Run und Zeitstempel organisiert sind.

4.3. Technologieauswahl

Bei der Auswahl der Technologien und Werkzeuge für die Implementierung des Systems wurden verschiedene Faktoren berücksichtigt, darunter die Leistungsfähigkeit, die Kompatibilität mit den Anforderungen und dessen Eignung für die spezifischen Anwendungsfälle.

Die konkrete Implementierung des Systems erfolgt in Python, da die für diese Arbeit benötigten Verfahren des maschinellen Lernens dort als ausgereifte Bibliotheken

vorliegen und sich Datenaufbereitung, Modellierung und Backend in derselben Sprache abbilden lassen, ohne Daten zwischen Laufzeitumgebungen übergeben zu müssen. Die Modelle selbst stammen aus drei Bibliotheken, die jeweils genau ein Verfahren beisteuern: **scikit-learn** den Random Forest, **lightgbm** das Gradient-Boosting-Verfahren und **PyTorch** das rekurrente Netz. **scikit-learn** stellt darüber hinaus die Evaluationsmetriken bereit, die für alle Verfahren einheitlich verwendet werden. Für die Datenaufbereitung werden **pandas** und **numpy** eingesetzt; die gruppenweisen Zeitreihenoperationen, aus denen die Merkmale entstehen (**groupby**, **shift**, **rolling**), sind dort unmittelbar vorhanden und müssen nicht selbst implementiert werden. Die Persistenz der Modelle übernimmt **joblib**, das gegenüber dem **pickle**-Modul der Standardbibliothek auf Objekte mit großen NumPy-Arrays zugeschnitten ist – genau der Fall bei trainierten Baum-Ensembles. Da der Datensatz **real_world_ev** im **.mat**-Format der Version 7.3 vorliegt, das intern HDF5 ist, wird **h5py** zum Einlesen verwendet. Das Backend nutzt **FastAPI** mit **pydantic** für die Request- und Response-Modelle sowie **uvicorn** als ASGI-Server; ausschlaggebend ist die Validierung der Schnittstellendaten anhand von Typannotationen, wodurch das Schema der API nicht getrennt vom Code gepflegt werden muss. Das Frontend wird mit **Angular** in Version 17 umgesetzt, wobei Standalone Components und Signals verwendet werden.

Die Begründung der Verfahrensauswahl selbst erfolgt in Abschnitt 3.5; technisch bedeutsam ist, dass die drei Architekturen in vier Konfigurationen implementiert sind, da das rekurrente Netz in zwei Eingabevarianten geführt wird. Beide Varianten teilen sich dieselbe Netzdefinition und Trainingsschleife und unterscheiden sich ausschließlich in der Aufbereitung der Eingabe, sodass der gemessene Unterschied zwischen ihnen tatsächlich auf die Repräsentation und nicht auf abweichende Implementierungsdetails zurückgeht. Die tabellarische Variante bezieht ihre Eingabe wie die Baum-Ensembles aus der Merkmalsmatrix, während die sequenzielle Variante den Datenrahmen selbst benötigt, um daraus gleitende Fenster zu bilden. Diese Unterscheidung ist an einem Attribut der jeweiligen Klasse ablesbar, anhand dessen Training und Evaluation den passenden Pfad wählen.

Eine bewusste Entwurfsentscheidung ist die verzögerte Einbindung optionaler Bibliotheken. **lightgbm** und **torch** gehören nicht zu den Pflichtabhängigkeiten des Projekts und werden erst dann importiert, wenn der zugehörige Klassifikator tatsächlich konstruiert wird. Ein Import auf Modulebene würde das gesamte Modul auf Systemen ohne diese Bibliotheken unbenutzbar machen, obwohl der betreffende Algorithmus überhaupt nicht angefordert wurde; der verzögerte Import hält den Standardpfad also lauffähig und verkürzt zugleich die Ladezeit, da insbesondere **torch** umfangreich ist. **scikit-learn** wird dagegen unmittelbar auf Modulebene geladen. Der Grund ist nicht, dass der Random Forest häufiger zum Einsatz käme –

bei der Wahl eines anderen Verfahrens wird er nie instanziiert –, sondern dass die Evaluationsmetriken derselben Bibliothek entstammen und unabhängig vom gewählten Algorithmus benötigt werden. `scikit-learn` ist zum Zeitpunkt der Modellkonstruktion folglich ohnehin geladen, sodass ein verzögerter Import keinen Vorteil brächte. Im Forecaster übernimmt eine Registry den verzögerten Import einheitlich für alle Algorithmen, einschließlich des Random Forest; dort steht nicht die Optionalität der Abhängigkeiten im Vordergrund, sondern die Erweiterbarkeit, da ein zusätzliches Verfahren lediglich einen Eintrag in der Registry erfordert und das Rahmenmodul keine Importliste pflegen muss.

Des Weiteren ist eine bewusste Entscheidung, den lokalen CPU für die Trainingsläufe zu verwenden. Die Modelle belegen keine unumengen an Speicher, sodass Trainingsläufe auch auf Standard-Hardware möglich sind. Die Trainingszeit ist länger als bei GPU-Beschleunigung, jedoch sind die Läufe Systemunabhängig und die Ergebnisse reproduzierbar, da die GPU-Implementierungen der Algorithmen nicht deterministisch sind. Die Trainingsläufe werden per API validiert und in einem abgekoppelten Thread ausgeführt, wodurch keine anderen Prozesse blockiert werden.

4.4. Umsetzung

`data_adapters.py` ist der Kern der Datenaufbereitung. Für jede Datenquelle gibt es einen Loader, der die Quelldaten in ein einheitliches Schema transformiert: `vehicle_id`, `timestamp`, `driving`. Da die Quellen unterschiedliche Formate besitzen, sowohl in Spalten als auch bei den Variablentypen, ist die Implementierung der Loader spezifisch für jede Quelle. Die Quellen werden dabei in Datentyp und Auflösung vereinheitlicht – ein lückenloses Stundenraster je Fahrzeug –, was für den Zeitursprung jedoch nicht gilt: Jede Quelle hat einen eigenen Epoch-Anker, der die relative Zeit in absolute Zeit umrechnet, was kein Problem darstellt, da der Zeitstempel ohnehin nicht in das Modell gefüttert wird. Bei VED etwa liegt der Zeitstempel als dezimalcodierte Tagesnummer (`DayNum`) vor, die über den quellenspezifischen Epoch-Anker in einen absoluten `datetime`-Zeitstempel umgerechnet und anschließend auf das Stundenraster gerundet wird (listing 4.1). Bei `yjmob` wird der Zeitstempel synthetisch auf den 01-01-2023 gesetzt und die Aktivität über Zellwechsel als Proxy für Bewegung abgeleitet. Bei `real_world_ev` wird die Aktivität aus dem Signal `in_use` über eine Ampere-Schwelle abgeleitet:

`lädt ≤ -0,5A < fährt nicht ≤ 5A < fährt.`

```
1 VED_EPOCH = pd.Timestamp("2017-11-01")    # DayNum = 1.0 ->
    dieser Tag
```



```

2 # DayNum: Dezimaltage seit dem Anker -> absoluter Zeitstempel
3 df["timestamp"] = VED_EPOCH + pd.to_timedelta(df["DayNum"] -
    1.0, unit="D")
4 df["timestamp"] = df["timestamp"].dt.floor("h")    #
    Stundenraster

```

Listing 4.1: Zeitanker der Quelle VED: relative Tagesnummer wird über einen festen Epoch-Anker in einen absoluten Zeitstempel überführt.

base.py enthält die Funktion `make_features`, die die Merkmalsmatrix aus dem Zeitreihen-Datenrahmen erstellt (listing 4.2). Enthalten sind Kalendermerkmale (Stunde, Wochentag, Wochenende), Lag-Merkmale (1, 2, 24 und 168 Stunden) sowie rollierende Mittelwerte über 24 und 168 Stunden. Das Array `FEATURE_COLS` legt die erwarteten Spalten fest und stellt sicher, dass alle Modellierungsläufe dasselbe Schema verwenden; eine zentrale Konfiguration vereinheitlicht zudem die Hyperparameter der Trainingsläufe. Ergänzend teilt die Funktion `split_by_time` den Datensatz zeitlich in Trainings- und Testdaten.

```

1 # alle Fenster je Fahrzeug -> kein Uebergriff ueber
    Fahrzeuggrenzen
2 g = df.groupby("vehicle_id", sort=False)["driving"]
3 df["lag_1"]    = g.shift(1)
4 df["lag_2"]    = g.shift(2)
5 df["lag_24"]   = g.shift(24)
6 df["lag_168"]  = g.shift(168)
7
8 # shift(1) VOR dem Fenster: der aktuelle Zielwert fliesst nie
9 # in sein eigenes Merkmal ein
10 shifted = g.shift(1)
11 df["rolling_mean_24"] = (
12     shifted.groupby(df["vehicle_id"]).rolling(24, min_periods
13         =1).mean()
14     .reset_index(level=0, drop=True))
15 df["rolling_mean_168"] = (
16     shifted.groupby(df["vehicle_id"]).rolling(168,
17         min_periods=1).mean()
18     .reset_index(level=0, drop=True))

```

Listing 4.2: Merkmalskonstruktion in `make_features` (gekürzt): alle Fenster gruppenweise je Fahrzeug, der rollierende Mittelwert erst nach `shift(1)`.

Der Schutz vor Data Leakage ergibt sich aus mehreren ineinandergreifenden Details, die in listing 4.2 sichtbar sind. *Erstens* blicken sämtliche Lag- und Roll-Fenster ausschließlich zurück; bei den Mittelwertsmerkmalen sorgt das vorgeschaltete `shift(1)` dafür, dass der aktuelle Zielwert nicht in seine eigene Berechnung einfließt. *Zweitens* sind alle Verschiebungen und Fenster über `groupby('vehicle_id')` auf das jeweilige Fahrzeug beschränkt, sodass keine Werte aus anderen Fahrzeugen übergreifen. *Drittens* trennt `split_by_time` an einem gemeinsamen Zeitpunkt statt fahrzeugweise, wodurch die Trainingsdaten zeitlich vollständig vor den Testdaten liegen. *Viertens* werden Zeilen mit noch undefinierten Lags – die ersten 168 Stunden je Fahrzeug – vor dem Training verworfen.

Kreuzvalidierung ist die zentrale Auswertung des Klassifikators. In `run_cross_validation.py` wird jede der fünf Datenquellen zeitlich in Trainings- und Testteil zerlegt. Anschließend wird pro Algorithmus für jede Quelle ein Modell trainiert und jedes Modell auf allen fünf Quellen getestet (listing 4.3). Das Ergebnis ist eine 5×5 -Isolationsmatrix, deren Zeilen die Trainingsquelle und deren Spalten die Testquelle darstellen. Die Diagonale misst die Güte innerhalb derselben Quelle, die übrigen Zellen die Übertragbarkeit der Modelle auf fremde Quellen.

```

1 for algo in algos:
2     models = {}
3     for name, (train_df, _) in splits.items():          #
4         Zeilen der Matrix
5         models[name] = train_model(train_df, cfg, algo=algo)
6
7     for train_src, model in models.items():
8         for test_src, (_, test_df) in splits.items():  #
9             Spalten der Matrix
10            metrics = evaluate(model, test_df)
11            metrics["model_trained_on"] = train_src
12            metrics["evaluated_on"]      = test_src

```

Listing 4.3: Aufbau der Isolationsmatrix (vereinfacht, ohne äußere Wiederholung)

Durch die vollständige Kreuzvalidierung lässt sich der Effekt der Datenquelle vom Effekt des Algorithmus trennen, denn der einzige Unterschied zwischen den Modellen einer Algorithmusspalte ist die Trainingsquelle. Die Merkmale werden dabei bewusst auf der vollständigen Zeitreihe berechnet und erst danach zeitlich getrennt, da andernfalls die wochenweiten Lag-Merkmale zu Beginn kurzer Testabschnitte durchweg undefiniert wären.

Forecasting und Quantile setzen sich aus dem Monte-Carlo-Rollout und der Unsicherheitsabschätzung zusammen. Neben dem Erwartungswert gibt der Forecaster ein unteres und oberes Quantil (P10/P90) aus. Das stündliche Band stammt aus einem Monte-Carlo-Rollout (listing 4.4): Über den Horizont werden mehrere Pfade fortgeschrieben, wobei der je Stunde gezogene Aktivitätszustand zur Vorgeschichte der nächsten Stunde wird. Dieser Modus ist die Voreinstellung des Auslieferungspfads; ein alternativer deterministischer Modus propagiert stattdessen die kontinuierliche Wahrscheinlichkeit und verzichtet bewusst auf das Band, da er bei sehr niedrigen Basisraten stabiler ist.

```

1 for h in range(horizon_h):
2     X = features_from(hist)                # Lags aus der
        bisherige Historie
3     probs = self.hourly_clf.predict_proba(X)[: , 1]
4     hist[:, idx] = rng.random(n) < probs    # 0/1 je Pfad
        ziehen -> wird Lag der Folgestunde
5 p10 = np.quantile(p_matrix, 0.1, axis=0)    # Quantile ueber
        die Pfad-Achse
6 p90 = np.quantile(p_matrix, 0.9, axis=0)

```

Listing 4.4: Autoregressiver Monte-Carlo-Rollout: Der je Stunde gezogene Zustand wird zur Vorgeschichte der nächsten; die Quantile entstehen über die Pfade.

Die Unsicherheit der Abfahrts- und Rückkehrzeit ist verfahrensspezifisch (listing 4.5): Der Random Forest leitet sie aus der Streuung der Einzelbäume ab, LightGBM aus einer Quantilsregression mit Pinball-Verlust.

```

1 # RandomForest: empirische Quantile ueber die Einzelbaeume
2 preds = np.array([tree.predict(X)[0] for tree in reg.
        estimators_])
3 p10, p90 = np.percentile(preds, 10), np.percentile(preds, 90)
4
5 # LightGBM: je Niveau ein eigener Regressor mit Pinball-
        Verlust
6 for alpha in (0.1, 0.5, 0.9):
7     LGBMRegressor(objective="quantile", alpha=alpha, **params
        )

```

Listing 4.5: Abfahrts-/Rückkehrzeit: RF nutzt die Streuung der Einzelbäume, LGBM eine Quantilsregression mit Pinball-Verlust.

4.5. Herausforderungen

Heterogene Rohformate und Zeitbasen stellen eine Herausforderung dar, da die Datenquellen unterschiedliche Formate und Zeitbasen aufweisen. Die Daten aus `Real_world_ev` liegen im `.mat`-Format vor und muss mittels der Python Bibliothek `h5py` gelesen werden, während die Daten aus `VED` und `yjmob` in CSV-Dateien gespeichert sind. Darüber hinaus ist die Zeitbasis der Datenquellen nicht einheitlich, was zusätzliche Schritte bei der Datenaufbereitung erfordert. Durch Interpolation und die Verwendung eines einheitlichen Zeitrasters wird diese Herausforderung gelöst.

Uneinheitliche Aktivitätssignale sind eine weitere Herausforderung, da die Definition der Aktivität je Quelle unterschiedlich ist. Während bei `VED` eine Aktivität nur dann vorliegt, wenn das Fahrzeug fährt, wird bei `Real_world_ev` eine Aktivität auch dann angenommen, wenn das Fahrzeug lädt. Bei `yjmob` wird die Aktivität über einen Proxy abgeleitet, der auf Zellwechseln basiert. Durch diese Unterschiede sind die Aktivitätssignale und deren zeittechnische Granularität nicht direkt vergleichbar. Eine Lösung besteht darin, die Aktivitätssignale auf ein einheitliches Zeitraster zu bringen und eine gemeinsame Definition der Aktivität zu verwenden, um die Vergleichbarkeit zu gewährleisten.

Degenerierte Fahrzeuge in der `VED`-Quelle stellen eine Herausforderung dar, da einige Fahrzeuge entweder 0% oder 100% Aktivität aufweisen. Fahrzeuge in dieser Quelle sind durch den `split_by_time` Algorithmus teilweise nur im Training- oder nur im Testset vorhanden, was zu einer Verzerrung der Ergebnisse führen kann. Eine Lösung besteht darin, diese Fahrzeuge auszuschließen oder zu filtern, um eine ausgewogenere Verteilung der Aktivitätssignale zu gewährleisten.

Rechenbeschränkungen werden in Form von CPU-only Training und langen Trainingszeiten deutlich. Die Trainingsläufe werden auf einer lokalen CPU durchgeführt, was zu längeren Trainingszeiten führt. Um die Laufzeit dennoch beherrschbar und die Läufe reproduzierbar zu halten, wird die Datenmenge an zwei Stellen bewusst begrenzt: Über die `--limit-*`-Flags lässt sich die Zahl der geladenen Fahrzeuge pro Quelle deckeln, und für das Training des rekurrenten Netzes wird die Zeilenzahl über eine feste Obergrenze (`max_train_samples`) beschränkt. Damit diese Begrenzung nicht die zeitliche Struktur zerstört, zieht `_subsample` eine mit festem Seed reproduzierbare, anschließend chronologisch sortierte Stichprobe der Zeilen (listing 4.6), sodass der zeitliche Validierungs-Tail ein Tail bleibt.

```
1 def _subsample(n: int, limit: int, seed: int) -> np.ndarray:
2     if n <= limit:                                # keine Deckelung
3         noetig
4         return np.arange(n)
5     rng = np.random.default_rng(seed)             # fester Seed ->
6         reproduzierbar
7     return np.sort(rng.choice(n, size=limit, replace=False))
```

Listing 4.6: Deckelung der Trainingszeilen; Seed hält den Lauf reproduzierbar, die Sortierung erhält die zeitliche Ordnung.

5. Evaluation

TODO: Einleitenden Text verfassen. Was wird evaluiert und mit welchem Ziel?

In der Evaluation wird beschrieben, wie die entwickelten Methoden aus Kapitel 3 auf die in Abschnitt 3.3 beschriebenen Daten angewendet werden und inwiefern die Ergebnisse den Erwartungen entsprechen. Die Evaluation dient dazu – durch festgelegte Ziele, Kriterien und Messgrößen (Abschnitt 5.1) – die Modelle in ihrer Fähigkeit zu bewerten, Fahraktivität zuverlässig vorherzusagen. Um das zu erreichen, werden die Modelle auf einer Testumgebung (Abschnitt 5.2) evaluiert und die Ergebnisse (Abschnitt 5.3) präsentiert. Die Evaluation umfasst sowohl quantitative Gütemaße als auch qualitative Analysen, um ein umfassendes Bild der Leistungsfähigkeit der Modelle zu erhalten.

5.1. Evaluationsdesign

TODO: Das Design der Evaluation beschreiben. Welche Metriken, Kriterien oder Hypothesen werden überprüft?

Durch verschiedene Darstellungen werden die Gütemaße der Modelle visualisiert, um die Genauigkeit, Stabilität und Verlässlichkeit der Vorhersagen zu bewerten (unter anderem Tabelle 5.1 und Tabelle 5.2). Anhand einer Kreuzvalidierungsmatrix (Abbildung 5.1, Abbildung A.1, Abbildung A.3 und Abbildung A.2) wird die Konsistenz der Vorhersagen über verschiedene Datenquellen hinweg überprüft. Da bei den verschiedenen Architekturen fünf Seeds existieren, werden die Gütemaße für die Stabilität gemittelt, um Randwerte zu minimieren. Die Gütemaße werden in Tabelle 5.2 für alle Modelle und Datenquellen tabellarisch dargestellt, um die Vergleichbarkeit zu erleichtern. Metriken die verwendet werden, sind Accuracy, F1-Score, ROC-AUC, PR-AUC und Brier-Score. Die Kennzahlen werden sowohl für die In-Distribution (auf derselben Datenquelle) als auch für die Out-of-Distribution (auf anderen Datenquellen) berechnet, um die Generalisierungsfähigkeit der Modelle zu bewerten. Accuracy wird als Negativbeispiel für die Limitierungen der Gütemaße diskutiert, da bei stark unausgeglichene Klassenverteilungen (wie in diesem Anwendungsfall)

die Accuracy eine fehlleitende Metrik sein kann. PR-AUC und Brier-Score werden als die wichtigsten Metriken hervorgehoben, da deren Fähigkeit bei unausgeglichenen Datensätzen die Qualität der Vorhersagen zu bewerten, besonders relevant ist. Im Laufe der Tests wird eine Ablation der Merkmalsgruppen durchgeführt, um die Bedeutung der einzelnen Merkmale für die Vorhersageleistung zu bewerten (Tabelle 5.3). Die Ablation untersucht, wie sich das Entfernen von bestimmten Merkmalsgruppen (Kalender, Lags, Rolling) auf die Metriken auswirkt.

5.2. Testumgebung

TODO: Die Testumgebung beschreiben (Hardware, Software, Datensätze, Konfigurationen).

Bei der Testumgebung handelt es sich um eine standardisierte Umgebung, die sicherstellt, dass die Evaluation reproduzierbar und konsistent durchgeführt werden kann. Zu den Bedingungen gehören folgende Dinge:

- **Hardware:** Die Tests werden auf einem lokalen Computer mit einer NVIDIA GeForce RTX 4070Ti Grafikkarte, 32 GB DDR5 RAM und einem Ryzen 7 9800X3D Prozessor durchgeführt. Die Hardware ist dafür spezialisiert, eine überschaubare Anzahl an anspruchsvollen Aufgaben in kurzer Zeit zu bewältigen, was für die Evaluation der Modelle von Vorteil ist.
- **Software:** Die Softwareumgebung besteht aus Visual Studio Code, Python 3.13 und den folgenden Technologien aus aktuellster Version zum 27.07.2026: scikit-learn, lightgbm, PyTorch, Pandas, Numpy, Angular und TypeScript.
- **Datensatzkonfiguration:** Jede Quelle wird chronologisch im Verhältnis 80/20 in Trainings- und Testteil geteilt. Der Umfang wird je Quelle begrenzt (**emobpy**: 30 Fahrzeuge, **ved**: 15 Fahrzeuge aus 4 Dateien, **yjmob100k**: 50 Personen; **real_world_ev** und **routine** vollständig), um die Laufzeit beherrschbar zu halten, ohne die Isolationsmatrix zu verändern.
- **Seeds, Parameter und Kontrollvariablen:** Jeder Lauf wird über fünf feste Zufalls-Seeds wiederholt und als Mittelwert samt Standardabweichung berichtet. Die Hyperparameter werden je Modell einmalig festgelegt und über die Isolations-experimente konstant gehalten, sodass sie zusammen mit dem Feature-Set als Kontrollvariablen wirken.

5.3. Ergebnisse

TODO: Die Ergebnisse der Evaluation präsentieren. Tabellen und Grafiken verwenden.

Bei der Gegenüberstellung des Random Forests mit dem naiven Baseline, zeigt sich in der Tabelle 5.1, dass sowohl bei der Metrik PR-AUC als auch bei der missgünstigen Metrik Accuracy, der Random Forest eine bessere Prognose liefert. Die Metrik Majority, die immer behauptet, dass ein Fahrzeug parkt, die Metrik Persistenz-168, welche die Vorwoche überträgt oder Klimatologie, die die Wahrscheinlichkeit zwischen dem Fahren zu einer bestimmten Uhrzeit, an einem Wochentag nimmt, schneiden alle nicht auf dem Niveau des Random Forest ab.

Tabelle 5.1.: Modell (RandomForest) im Vergleich zu naiven Baselines, In-Distribution je Datenquelle auf demselben zeitlichen Test-Split. Majority = „immer geparkt“; Persistenz-168 = gleiche Stunde der Vorwoche; Klimatologie = $P(\text{fahren} \mid \text{Wochentag}, \text{Stunde})$ aus dem Trainingsteil.

(a) PR-AUC (Zufalls-Baseline = Aktiv-Rate; höher = besser)

Datenquelle	Modell (RF)	Majority	Persistenz-168	Klimatologie
emobpy	0,267	0,096	0,105	0,172
real_world_ev	0,640	0,023	0,023	0,022
ved	0,091	0,048	0,078	0,083
routine	0,884	0,264	0,672	0,797
yjmob100k	0,683	0,382	0,478	0,489

(b) Accuracy (Majority schneidet durch hohe Aktivitätsrate sehr gut ab)

Datenquelle	Modell (RF)	Majority	Persistenz-168	Klimatologie
emobpy	0,697	0,904	0,838	0,904
real_world_ev	0,983	0,977	0,956	0,977
ved	0,880	0,952	0,919	0,952
routine	0,898	0,736	0,887	0,868
yjmob100k	0,726	0,618	0,659	0,619

Abbildung 5.1 stellt die Kreuzvalidierung des Driving-Classifiers mit dem Random Forest Algorithmus dar. Die Matrix zeigt die F1-Scores für jede Kombination von Trainings- und Testdatenquelle. Die Diagonale der Matrix repräsentiert

die In-Distribution-Leistung, während Metriken außerhalb der Diagonale die Out-of-Distribution-Leistung darstellen. Die Ergebnisse zeigen, dass die Modelle sich gegenüber dem naive Baseline (majority, persistenz-168, klimatologie) deutlich verbessern. Eine Zeile steht besonders hervor; VED zeigt unter den Datensätzen die geringste Vorhersageleistung, wobei die Datenmenge und Aktivitätsrate mehr als ausreichend sind (Tabelle 5.2). Im Anhang in Abschnitt A.1 befinden sich weitere Kreuzvalidierungsmatrizen mit den übrigen verwendeten Algorithmen.

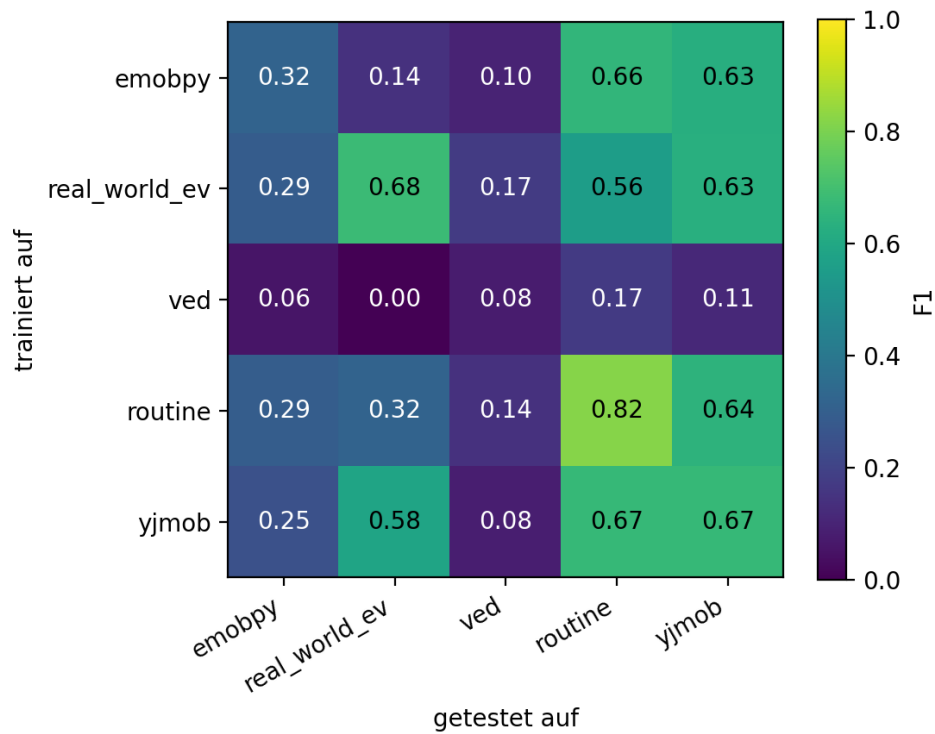


Abbildung 5.1.: 5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (RandomForest)

TODO: lstm matrizen ansprechen

Tabelle 5.2.: In-distribution (in eigener Menge – Datenquelle, in diesem Fall) Güte-
maße des Driving-Classifiers je Datenquelle. RandomForest, LightGBM
und tab. LSTM-Variante nutzen Merkmalssatz; seq. Variante sieht
Rohfenster. „n. z.“ = nicht als Trainingsquelle zugelassen, da Quelle
Mindestkriterium von 100 Fahrzeugwochen verfehlt (Abschnitt 3.5).

(a) RandomForest

Datenquelle	Aktiv-Rate	Acc.	F1	ROC	PR	Brier
emobpy	0,096	0,697	0,319	0,798	0,267	0,169
real_world_ev	0,023	0,983	0,681	0,884	0,640	0,020
ved	0,048	0,880	0,077	0,733	0,091	0,090
routine	0,264	0,898	0,818	0,961	0,884	0,073
yjmob100k	0,382	0,726	0,669	0,798	0,683	0,182

(b) LightGBM

Datenquelle	Aktiv-Rate	Acc.	F1	ROC	PR	Brier
emobpy	0,096	0,641	0,307	0,809	0,277	0,191
real_world_ev	0,023	0,969	0,366	0,850	0,256	0,027
ved	0,048	0,846	0,092	0,687	0,077	0,110
routine	0,264	0,880	0,783	0,954	0,873	0,090
yjmob100k	0,382	0,726	0,676	0,807	0,699	0,180

(c) LSTM, tabellarisch (identischer Merkmalssatz)

Datenquelle	Aktiv-Rate	Acc.	F1	ROC	PR	Brier
emobpy	0,096	0,610	0,286	0,780	0,255	0,205
real_world_ev	0,023			n. z.		
ved	0,048			n. z.		
routine	0,264			n. z.		
yjmob100k	0,382	0,726	0,676	0,806	0,703	0,180

(d) LSTM, sequenziell (Fenster der letzten 168 Stunden)

Datenquelle	Aktiv-Rate	Acc.	F1	ROC	PR	Brier
emobpy	0,096	0,661	0,323	0,820	0,288	0,194
real_world_ev	0,023			n. z.		
ved	0,048			n. z.		
routine	0,264			n. z.		
yjmob100k	0,382	0,719	0,657	0,789	0,670	0,186

TODO: memo ablation-interpretation (Tabelle 5.3): (1) lags = tragende gruppe – 'ohne Lags' kostet am meisten (real_world_ev -0,552, routine -0,161, yjmob -0,109); 'nur Lags' erreicht fast/über referenz. (2) rolling redundant – 'ohne Rolling' bei 3/5 quellen BESSER als voll (real_world_ev 0,705>0,619, ved 0,100>0,089, routine 0,892>0,886); 'nur Rolling' durchweg am schlechtesten -> rolling bringt gegenüber lags nur rauschen. (3) kalender quellenabhängig: bei routine (regelbasiert) reicht 'nur Kalender' fast (0,803 vs 0,886), bei real_world_ev bricht es ein (0,018). LIMITATION: ved durchgehend nahe zufall (aktiv-rate 0,048) -> deltas dort nicht interpretierbar; real_world_ev n=1 fahrzeug -> lgbm schwankt 0,177..0,669 = stichprobenrauschen.

Tabelle 5.3.: Ablation der Merkmalsgruppen: PR-AUC je Datenquelle, zeitlich in-distribution. Verworfenen Zeilen richten sich in allen Varianten nach dem vollständigen Merkmalssatz, sodass je Quelle identische Beobachtungen zugrunde liegen.

(a) RandomForest					
Variante	emobpy	real_world_ev	ved	routine	yjmob100k
voll (Referenz)	0,268	0,619	0,089	0,886	0,683
ohne Kalender	0,230	0,621	0,064	0,795	0,660
ohne Lags	0,207	0,067	0,078	0,724	0,574
ohne Rolling	0,249	0,705	0,100	0,892	0,667
nur Kalender	0,173	0,018	0,087	0,803	0,487
nur Lags	0,207	0,669	0,094	0,793	0,670
nur Rolling	0,134	0,187	0,048	0,253	0,497

(b) LightGBM					
Variante	emobpy	real_world_ev	ved	routine	yjmob100k
voll (Referenz)	0,277	0,177	0,074	0,861	0,698
ohne Kalender	0,238	0,308	0,064	0,752	0,683
ohne Lags	0,216	0,068	0,077	0,825	0,598
ohne Rolling	0,261	0,625	0,122	0,871	0,690
nur Kalender	0,173	0,019	0,095	0,820	0,487
nur Lags	0,207	0,669	0,091	0,792	0,670
nur Rolling	0,135	0,183	0,049	0,263	0,517

(c) LSTM, tabellarischs					
Variante	emobpy	real_world_ev	ved	routine	yjmob100k
voll (Referenz)	0,259	0,504	0,099	0,582	0,701
ohne Kalender	0,233	0,358	0,101	0,704	0,692
ohne Lags	0,201	0,088	0,087	0,353	0,603
ohne Rolling	0,256	0,567	0,101	0,598	0,691
nur Kalender	0,163	0,018	0,079	0,362	0,480
nur Lags	0,206	0,614	0,087	0,756	0,666
nur Rolling	0,127	0,152	0,048	0,253	0,526

6. Diskussion

TODO: Einleitenden Text verfassen. Was wird in diesem Kapitel diskutiert?

6.1. Interpretation der Ergebnisse

TODO: Die Ergebnisse aus Kapitel 5 interpretieren. Was bedeuten die Ergebnisse im Kontext der Forschungsfragen?

6.2. Vergleich mit verwandten Arbeiten

Da sich bislang nur wenige Arbeiten mit der Erstellung von Modellen für die Prognose individuellen künftigen Mobilitätsverhaltens befasst haben, liegt keine Grundlage für einen direkten Vergleich vor, sodass hierzu keine belastbare Aussage getroffen werden kann.

Studien die sich mit individuellem Mobilitätsverhalten befassen, schauen hauptsächlich nach *State of Charge* oder *Ortsdaten*. **TODO:** Wie verhalten sich die Ergebnisse im Vergleich zu verwandten Arbeiten aus Abschnitt 2.4.1? – – verbessern

6.3. Limitationen

TODO: Welche Einschränkungen hat die Arbeit? Was konnte nicht untersucht werden? Welche Annahmen wurden getroffen? – – verbessern

- aktive datenerhebung
- realer verkehr
- parallele studien

6.4. Implikationen

TODO: Welche praktischen und theoretischen Implikationen ergeben sich aus den Ergebnissen?

7. Fazit und Ausblick

7.1. Beantwortung der Forschungsfragen

TODO: Jede Forschungsfrage aus Abschnitt 1.4 explizit beantworten.

1. **TODO:** Antwort auf Forschungsfrage 1
2. **TODO:** Antwort auf Forschungsfrage 2
3. **TODO:** Antwort auf Forschungsfrage 3

7.2. Ausblick

TODO: Welche zukünftigen Arbeiten könnten auf dieser Arbeit aufbauen? Welche offenen Fragen bleiben bestehen?

Literaturverzeichnis

- Ali, Mohammad Wazed, Asif bin Mustafa, Md. Auferul Moin Shuvo und Bernhard Sick (2024). *Location based Probabilistic Load Forecasting of EV Charging Sites: Deep Transfer Learning with Multi-Quantile Temporal Convolutional Network*. DOI: 10.48550/arXiv.2409.11862. arXiv: 2409.11862 [cs.LG].
- Amara-Ouali, Yvenn, Bachir Hamrouche, Guillaume Principato und Yannig Goude (2025). „Quantifying the Uncertainty of Electric Vehicle Charging with Probabilistic Load Forecasting“. In: *World Electric Vehicle Journal* 16.2, S. 88. DOI: 10.3390/wevj16020088.
- Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. New York: Springer.
- Breiman, Leo, Jerome H. Friedman, Richard A. Olshen und Charles J. Stone (1984). *Classification and Regression Trees*. Belmont, CA: Wadsworth.
- dida (2026). *Was ist ein LSTM neuronales Netzwerk?* Abbildung. dida Datenschmiede GmbH. URL: <https://dida.do/de/was-ist-ein-lstm-neuronales-netzwerk> (besucht am 09.07.2026).
- Gaete-Morales, Carlos, Hendrik Kramer, Wolf-Peter Schill und Alexander Zerrahn (2020). *emobpy: A Tool for Generating Battery Electric Vehicle Time Series*. Synthetische BEV-Profil auf Basis deutscher Mobilitätsstatistik (MiD). DOI: 10.5281/zenodo.3931663.
- (2021). „An open tool for creating battery-electric vehicle time series from empirical data, emobpy“. In: *Scientific Data* 8.1. Peer-reviewtes Paper zum emobpy-Tool; Begleitsoftware siehe Gaete-Morales, Kramer, Schill und Zerrahn 2020, S. 152. DOI: 10.1038/s41597-021-00932-9. URL: <https://www.nature.com/articles/s41597-021-00932-9> (besucht am 15.07.2026).
- Géron, Aurélien (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3. Aufl. Sebastopol, CA: O'Reilly Media.
- Hastie, Trevor, Robert Tibshirani und Jerome Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. Aufl. New York: Springer. DOI: 10.1007/978-0-387-84858-7.
- Hyndman, Rob J. und George Athanasopoulos (2021). *Forecasting: Principles and Practice*. 3. Aufl. Melbourne: OTexts. URL: <https://otexts.com/fpp3/> (besucht am 15.06.2026).

- Ji u. a. (2023). „Rethinking the Regularity in Mobility Patterns of Personal Vehicle Drivers: A Multi-city Comparison Using a Feature Engineering Approach“. In: *Transactions in GIS* 27.3, S. 663–685. DOI: 10.1111/tgis.13043.
- Kaufman, Shachar, Saharon Rosset, Claudia Perlich und Ori Stitelman (2012). „Leakage in Data Mining: Formulation, Detection, and Avoidance“. In: *ACM Transactions on Knowledge Discovery from Data* 6.4, S. 1–21. DOI: 10.1145/2382577.2382579.
- Koenker, Roger und Gilbert Bassett (1978). „Regression Quantiles“. In: *Econometrica* 46.1, S. 33–50. DOI: 10.2307/1913643.
- Kumar, Raman und Anuj Jain (2023). „Driving behavior analysis and classification by vehicle OBD data using machine learning“. In: *The Journal of Supercomputing* 79.16, S. 18800–18819. DOI: 10.1007/s11227-023-05364-3.
- Lindroth, Tobias, Axel Svensson, Niklas Åkerblom, Mitra Pourabdollah und Morteza Haghir Chehreghani (2024). „Online Learning Models for Vehicle Usage Prediction During COVID-19“. In: *IEEE Transactions on Intelligent Transportation Systems*. DOI: 10.1109/TITS.2024.3361676.
- Meinshausen, Nicolai (2006). „Quantile Regression Forests“. In: *Journal of Machine Learning Research* 7, S. 983–999. URL: <https://www.jmlr.org/papers/v7/meinshausen06a.html>.
- Oh, Geunseob, David J. LeBlanc und Huei Peng (2022). „Vehicle Energy Dataset (VED), A Large-scale Dataset for Vehicle Energy Consumption Research“. In: *IEEE Transactions on Intelligent Transportation Systems* 23.4. Daten: <https://github.com/gsoh/VED>, S. 3302–3312. DOI: 10.1109/TITS.2020.3035596.
- others, and (2025). „Individual Mobility Prediction by Considering Current Traveling Features and Historical Activity Chain“. In: *Geo-spatial Information Science*, S. 2988–3015. DOI: 10.1080/10095020.2025.2455005.
- Pozzato, Gabriele, Anirudh Allam und Simona Onori (2023). *Data from: Analysis and Key Findings from Real-World Electric Vehicle Field Data*. Reale 1-Jahres-Felddaten eines Audi e-tron (CAN-Bus, SOC). DOI: 10.17632/7vdkzpnjgj.2. URL: <https://data.mendeley.com/datasets/7vdkzpnjgj/2> (besucht am 13.06.2026).
- ResearchGate (2021). *Flow Diagram of Gradient Boosting Machine Learning Method*. Abbildung. URL: https://www.researchgate.net/figure/Flow-diagram-of-gradient-boosting-machine-learning-method-The-ensemble-classifiers_fig1_351542039 (besucht am 02.07.2026).
- (2022). *Diagram of the Bagging Technique Used for Classification or Regression Problems in ML*. Abbildung. URL: https://www.researchgate.net/figure/Diagram-of-the-bagging-technique-used-for-classification-or-regression-problems-in-ML_fig1_362065998 (besucht am 02.07.2026).

-
- Rijsbergen, C. J. van (1979). *Information Retrieval*. 2. Aufl. Kap. 7, Definition des F_β /E-Maßes. London: Butterworths.
- Sanami, Saba, Hesam Mosalli, Yu Yang, Hen-Geul Yeh und Amir G. Aghdam (2025). *Demand Forecasting for Electric Vehicle Charging Stations using Multivariate Time-Series Analysis*. DOI: 10.48550/arXiv.2502.16365. arXiv: 2502.16365 [cs.LG].
- Schilli, Michael (2017). „KI lernt mit Entscheidungsbaeumen“. In: *Linux-Magazin* 08. URL: <https://www.linux-magazin.de/ausgaben/2017/08/snapshot/> (besucht am 13.06.2026).
- ScienceDirect (2026). *Light Gradient Boosting Machine*. Abbildung. ScienceDirect Topics, Engineering. URL: <https://www.sciencedirect.com/topics/engineering/light-gradient-boosting-machine> (besucht am 03.07.2026).
- Su, Zhihan, Xiaochen Liu, Hao Li, Tao Zhang, Xiaohua Liu und Yi Jiang (2025). „A vehicle trajectory-based parking location recognition and inference method: Considering both travel action and intention“. In: *Sustainable Cities and Society* 119, S. 106088. DOI: 10.1016/j.scs.2024.106088.
- Wuttke, Laurenz (2024). *Machine Learning: Algorithmen, Methoden und Beispiele*. Datasolut GmbH. URL: <https://datasolut.com/was-ist-machine-learning/> (besucht am 13.06.2026).
- Yabe, Takahiro, Kota Tsubouchi, Toru Shimizu, Yoshihide Sekimoto, Kaoru Sezaki, Esteban Moro und Alex Pentland (2024). „YJMob100K: City-scale and longitudinal dataset of anonymized human mobility trajectories“. In: *Scientific Data* 11.1, S. 397. DOI: 10.1038/s41597-024-03237-9.

A. Anhang

A.1. Zusätzliche Abbildungen

Die Kreuzvalidierungsmatrix des LightGBM Modells (Abbildung A.1) zeigt, wie auch die Kreuzvalidierungsmatrix des Random Forest Modells (Abbildung 5.1), die F1-Scores für jede Kombination von Trainings- und Testdatenquelle.

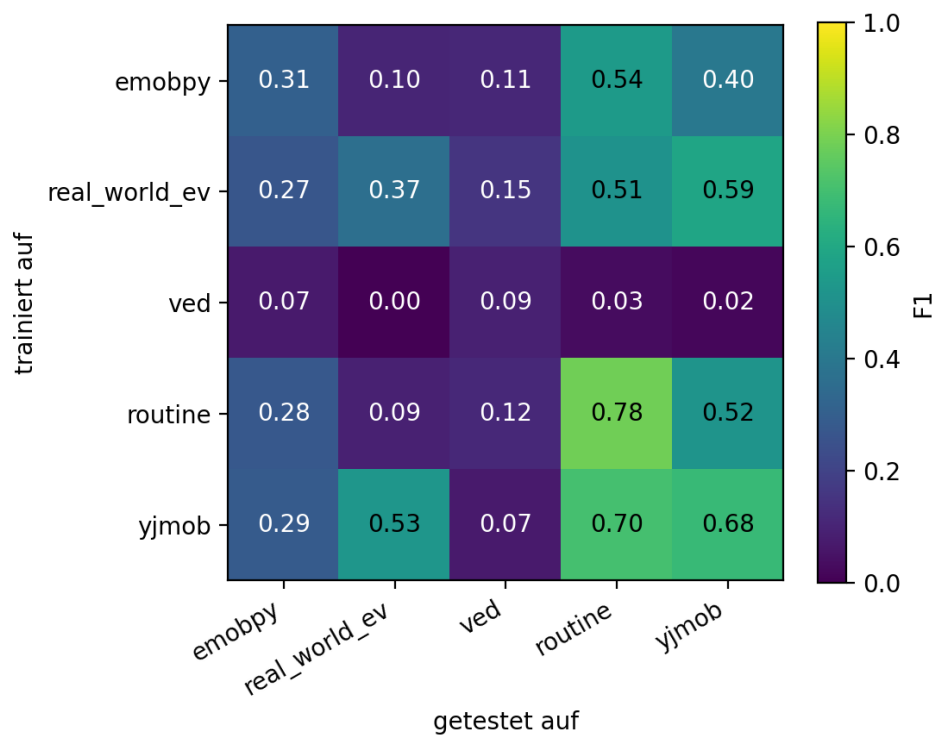


Abbildung A.1.: 5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (LightGBM)

Die Kreuzvalidierungsmatrizen der beiden LSTM-Varianten ergänzen die im Ergebnisteil (Abbildung 5.1, Abbildung A.1) gezeigten Matrizen von Random Forest und LightGBM. Sie umfassen nur **emobpy** und **yjmob100k** als Trainingsquellen, da die übrigen Quellen den in Abschnitt 3.5 festgelegte Mindestumfang von 100 Fahrzeugwochen unterschreiten, wobei alle fünf Datensätze zum Testen verwendet wurden.

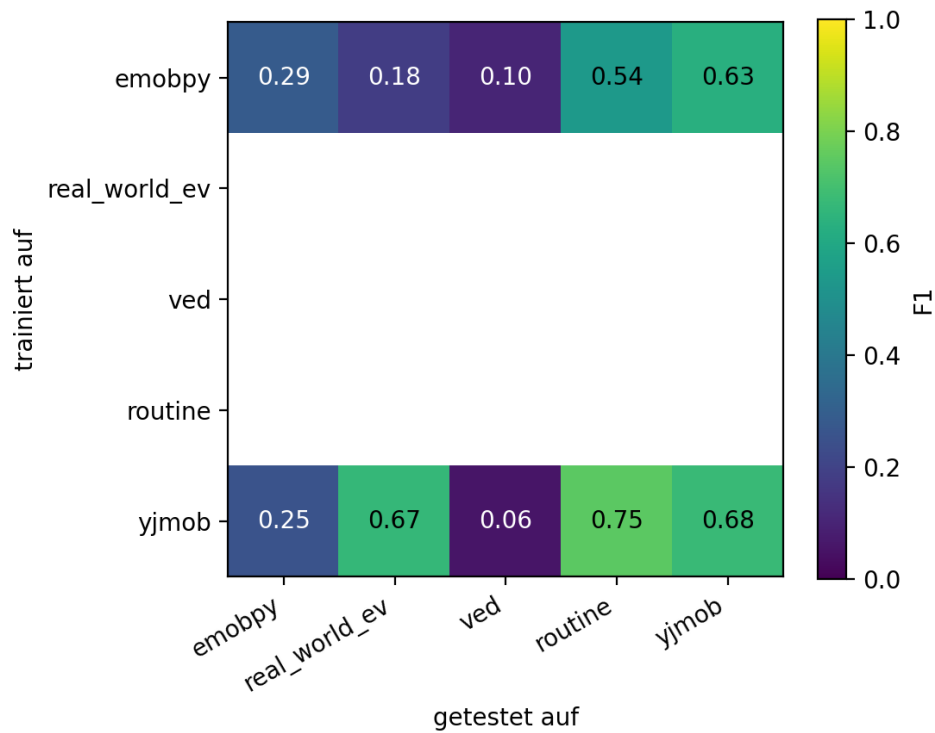


Abbildung A.2.: 5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (LSTM, tabellarische Repräsentation)

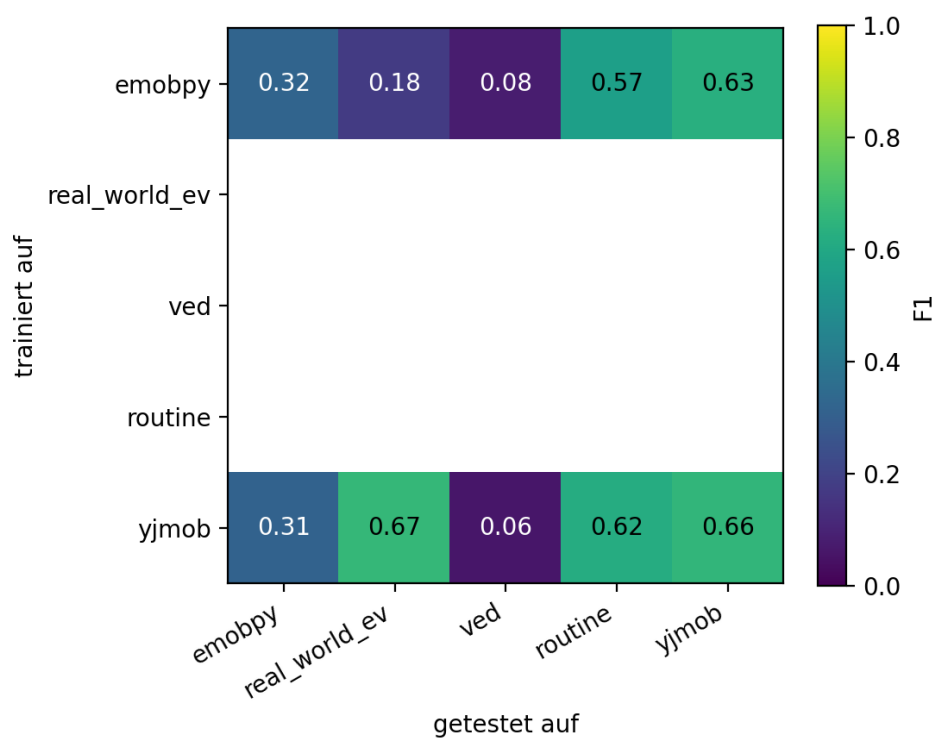


Abbildung A.3.: 5×5-Kreuzvalidierungsmatrix des Driving-Classifiers (LSTM, sequentielle Repräsentation)

A.2. Zusätzliche Tabellen

TODO: Zusätzliche Tabellen hier einfügen, falls vorhanden.

A.3. Quellcode

TODO: Relevanten Quellcode hier einfügen, falls vorhanden.

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Masterarbeit mit dem Titel

*Prognose individuellen Mobilitätsverhaltens auf Basis eines mit Bewegungsdaten
trainierten Machine-Learning-Modells*

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Schriften entnommen sind, sind als solche kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

.....

Ort, Datum

.....

Unterschrift